

컴파일러 수업 정리 노트 (Ch1~3)

Ch1. Introduction — 컴파일러 개론

1.1 컴파일러의 정의

컴파일러 = 번역기(Translator). 소스 언어(source language)로 작성된 입력 문장들을 타겟 언어(target language)의 동등한(equivalent) 문장으로 변환하는 프로그램이다.

1.2 프로그램 실행의 세 가지 방식

(1) 컴파일러 방식

소스 프로그램(C/C++)을 타겟 프로그램(기계어 실행 파일, executable)으로 변환 후, HW(CPU)에서 직접 실행한다. Compile time과 Runtime이 분리된다.

[C/C++ source]		[machine-language executable]
for (j=0; j<i; j++) {	컴파일	\$006
if (A[j]>A[j+1]) {	----->	LDWS -4(%r23),%r25
temp = A[j];		LDWS,MA 4(%r23),%r26
A[j] = A[j+1];		COMB,<= %r25,%r26,\$007
A[j+1] = temp;		STWS %r26,-8(%r23)
}		STWS %r25,-4(%r23)
}		\$007
		ADDIB,<,N 1,%r24,\$006+4
		LDWS -4(%r23),%r25

(2) 인터프리터(Virtual Machine) 방식

별도의 컴파일 없이, 인터프리터(SW)가 소스 프로그램을 직접 실행한다. Python, JavaScript 등. compile time 없이 runtime만 존재. 컴파일 방식보다 느리지만 개발이 빠르고 여러 핸들링과 이식성(portability)이 좋아 현재 주류(mainstream)이다.

(3) 하이브리드 방식 (Compiler + Virtual Machine)

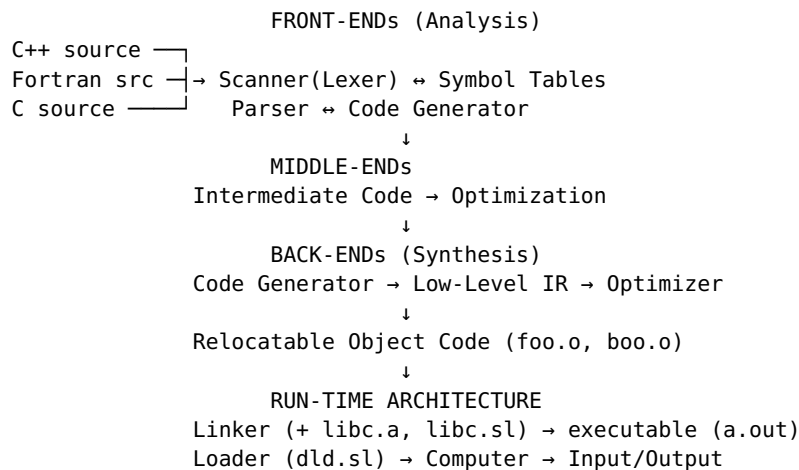
Java가 대표. 소스를 IR(바이트코드)로 컴파일한 후, VM에서 실행한다. VM은 인터프리터 또는 JIT(Just-In-Time) 컴파일러를 사용한다. 이식성과 성능을 동시에 추구.

[Java source] → 컴파일러 → [bytecode IR] → VM(인터프리터/JIT) → 결과
compile time runtime

```
바이트코드 예시: sipush 1000 / newarray int / astore_1 / iconst_0 /
istore 2 / goto 18 / ...
```

1.3 현대 컴파일러의 구조

컴파일은 (1) 소스 언어의 분석(analysis)과 (2) 타겟 언어의 합성(synthesis)으로 이루어진다.



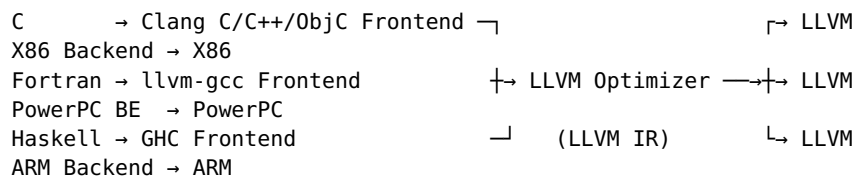
Front-end (분석): - Lexical analysis (어휘 분석) - Syntactic analysis (구문 분석) - Semantic analysis (의미 분석)

Back-end (합성): - IR 생성: P-code, U-code, bytecode, parse tree, AST, register-based IR 등 - Machine-independent optimization (기계 독립 최적화) - Machine code generation and optimization (기계 종속 코드 생성 및 최적화)

Runtime architecture: Linking, Loading, Shared libraries (DLL)

LLVM (Low Level Virtual Machine)

모듈화되고 재사용 가능한 컴파일러 및 툴체인 모음. GCC(GNU Compiler Collection)와 유사.



3단계 구조: Source Code → Frontend → **LLVM IR** → Optimizer → **LLVM IR** → Backend → Machine Code

1.4 프론트엔드 복습

Lexical Analysis (Lexer)

입력 문자 스트림을 하나씩 읽어 사전 정의된 토큰(token)을 식별한다. 정규 표현식(Regular Expression)으로 토큰 패턴을 정의하고, 유한 오토마타(Finite Automata) 기반 Lexer를 자동 생성한다.

도구: `lex foo.l; cc lex.yy.c -ll; a.out < test.c;`

lex 파일 구조 — (정규 표현식, 액션) 쌍:

```

Letter  [a-zA-Z]
Digit   [0-9]
Id       {Letter}({Letter}|{Digit})*
%%
{Id}     {yylval = install_id(); return(ID); }
%%
  
```

```
install_id { ... }
```

Syntax Analysis (Parser)

토큰에서 문법 구조를 복원하여 파스 트리(parse tree)를 구성한다. 문맥 자유 문법(Context-Free Grammar)으로 문법을 표현한다.

프로덕션(production) 예시:

```
var_decl : type ID ";"
         | type ID "[" const_expr "]" ";"
type     : type_id | struct_specifier
type_id  : ID
```

int x ; → 파스 트리: var_decl → type(type_id(ID:"int")) + ID:"x" + ";"

파싱 방식: - Top-down parsing: 시작 심볼에서 아래로 확장 (도구: Ometa) - Bottom-up parsing: 앞에서 시작 심볼로 구축 — 더 강력함 (도구: yacc)

도구를 사용해 파서를 자동 생성한다 → “컴파일러는 컴퓨터 과학의 승리 (triumph)”

Semantic Analysis (의미 분석)

문맥 자유 문법으로 표현 불가능한 규칙을 검사한다: - 변수를 사용 전에 선언했는가 (e.g., int x; ..., x = 10;) - 실인자와 형식인자의 개수가 일치하는가 - 타입이 일치하는가 (e.g., x = y + z;에서 정수+문자열 방지)

심볼 테이블(symbol table)을 구축하여 분석한다. 자동 생성 불가 — 직접 구현 필요. 효율을 위해 파싱 도중 의미 분석과 코드 생성을 동시에 수행할 수 있다.

파싱 중 의미 분석 예시 (yacc 문법+액션):

```
var_decl : type ID ";" { declare($2, makevardecl($1)); }
unary    : INT-CONST { $$ = makenumconstdecl(inttype, $1); }
         | ID        { $$ = findcurrentdecl($1); }
binary   : unary     { $$ = $1->type; }
         | binary '+' binary { $$ = plustype($1, $3); }
assignment : unary "=" expr { check_isvar($1);
check_compatible($1,$3); $$ = $1->type; }
```

Intermediate Representation (IR)

IR의 종류: - **Parse tree / AST**: 트리 구조 - **Abstract stack machine**

code: push a; push b; add; store c - x = y; (x: 전역, y: 지역)의 스택코드: push_const Lglob+4, push_reg SP, push_const 1, add, fetch, assign -

Register-based intermediate code: 가상 레지스터 사용

스택 머신 코드의 장점: 코드 생성이 단순하고, 코드 크기가 작다.

1.5 컴파일러 최적화

최적화의 정의

계산을 동등하지만 더 나은(equivalent but better) 계산으로 변환하는 것

실행 시간 공식

실행 시간 = Instruction Count × CPI × Cycle Time

- Instruction Count (IC): 실행되는 명령어의 총 수
- CPI (Cycles Per Instruction): 명령어 하나당 평균 클럭 사이클 수
- Cycle Time: 클럭 한 사이클의 시간 (HW에 의해 결정)

컴파일러가 줄일 수 있는 것: **IC**와 **CPI** (Cycle Time은 HW의 몫)

구체적 최적화 기법 목록

- 코드 내 명령어 수 감소
- 레지스터 할당 개선
- 독립 명령어 그룹핑 (슈퍼스칼라 병렬 실행)
- 비싼 명령어를 싼 명령어로 교체 (e.g., multiply → add/shift)
- 루프를 더 밀도 있게(denser) 만들
- 멀티코어 병렬화
- 캐시 미스 감소

메모리 계층별 지연 시간 (Numbers Everyone Should Know)

L1 cache reference	0.5 ns	
Branch mispredict	5 ns	
L2 cache reference	10 ns	
Mutex lock/unlock	100 ns	
Main memory reference	100 ns	← L1 대비 200배
Compress 1KB (zippy)	10,000 ns	
Send 2KB over 1Gbps	20,000 ns	
Read 1MB from memory	250,000 ns	
RTT within datacenter	500,000 ns	
1 disk seek	10,000,000 ns	← L1 대비 2천만배
Read 1MB from disk	30,000,000 ns	

→ 컴파일러가 데이터를 레지스터/캐시에 잘 유지하도록 최적화하면 성능 차이가 극적.

최적화가 ISA 설계에 미치는 영향

IBM PowerPC, HP PA-RISC 등의 ISA에는 컴파일러 최적화를 전제로 설계된 **복합 명령어(compound instruction)**가 있다.

예: 배열 초기화 for (i=0; i<N; i++) A[i]=0;의 최적화된 어셈블리:

```
LD0    -440(%r30),%r31      ; &A
LDI     -100,%r23            ; 카운터 = -100
$003
ADDIB,< 1,%r23,$003         ; 카운터++, 카운터<0이면 $003으로
STWM    %r0,100(%r31)       ; 0 → A[i], 포인터 += 100
```

→ 루프 본체가 단 **2개 명령어**. ADDIB,<는 증가+비교+분기를 하나의 명령어로, STWM은 저장+포인터증가를 하나의 명령어로 수행한다.

최적화 컴파일러의 페이지 구조

페이지별로 순차 진행하며, 각 페이지를 지날수록 코드 품질 개선. 서랍장처럼 페이지 추가/삭제 용이.

Graph Representation of Intermediate Code

↓

- (1) Basic Block Optimization
- (2) Dataflow Analysis + Interval Analysis

- (3) Global Common Subexpression Elimination
 - (4) Promotion of Memory Operations
 - (5) Loop Invariant Code Motion
 - (6) Induction Variable Analysis
 - (7) Register Reassociation
 - (8) Register Web (Live Range) Builder
 - (9) Instruction Scheduling ← pre-Register Allocation
 - (10) Register Allocation ←
 - (11) Peephole Optimization
 - (12) Instruction Scheduling ← post-Register Allocation
- ↓

Graph Representation of Optimized Code

최적화 레벨: - -01: 기본 최적화만 - -02 (= -0): 안정적인 최적화 - -0x (x>2): 공격적이지만 항상 안정적이지는 않음

Bubble Sort 최적화 전/후 비교 (HP PA-RISC)

```
#define N 100
main() {
    int A[N], i, j, temp;
    for (i = N-1; i >= 0; i--)
        for (j = 0; j < i; j++)
            if (A[j] > A[j+1]) {
                temp = A[j]; A[j] = A[j+1]; A[j+1] = temp;
            }
}
```

비최적화 코드 (inner loop): 약 50줄 어셈블리. 변수 i, j, temp를 매번 메모리 (스택)에서 읽고 쓰고, 배열 주소(&A)를 매번 재계산(LDO -448(%r30)).

최적화 코드 (inner loop):

```
$006
LDWS    -4(%r23),%r25          ; A[j]
LDWS,MA 4(%r23),%r26          ; A[j+1], 포인터 자동 증가
COMB,<=,N %r25,%r26,$007      ; A[j] <= A[j+1]이면 swap 건너뛰
STWS    %r26,-8(%r23)         ; A[j+1] → A[j]
STWS    %r25,-4(%r23)         ; A[j] → A[j+1]
$007
ADDIB,<,N 1,%r24,$006+4       ; j++, j<i이면 $006으로
LDWS    -4(%r23),%r25          ; 다음 반복의 A[j] 미리 로드
```

→ 모든 변수가 레지스터에, 복합 명령어 사용, 루프가 극적으로 축소.

최적화 컴파일러의 문제 해결 방법론

1. 빈번한 케이스(common cases)를 찾아라
2. 수학적으로 정형화(formulate mathematically)하라
3. 적절한 페이즈를 결정하라
4. 알고리즘을 개발하라
5. 구현하라
6. 실제 데이터로 평가하라

“케이스가 거의 없는 최적화에는 절대 시간을 쓰지 마라”

1.6 사례 연구: DRAM 테스트 패턴 프로그램 변환

배경: ATE(Automatic Test Equipment)는 ALPG(Algorithmic Pattern Generator)를 사용해 DRAM(DDR4 등)을 테스트한다. 각 클럭마다 패턴(command + data + address 비트 벡터)을 전송한다. 문제: ATE 장비 제조사마다 프로그래밍 언어가 다르고, 같은 장비의 다른 모델 간에도 HW(ALPG 개수, 레지스터 수)가 다르다.

해결: 통합 IR 기반 Translation Platform — 컴파일러와 동일한 구조 적용

[장비A 프로그램 TEST.asc] → 분석(Analysis) → [통합 IR] → 합성(Synthesis) → [장비B 프로그램 TEST.cs]
[장비A2 프로그램 TEST.asc2] →
→ [장비B2 프로그램 TEST.cs2]

Frontend: Lexing, Parsing, IR 생성, 인터프리터/VM. Backend: 불규칙 레지스터 할당, 다른 속도 DRAM 스케줄링, 가독성 중심 컴파일, 변환 검증, HW 설정 파일 자동 생성.

→ 컴파일러 기법이 프로그래밍 언어 번역을 넘어 다양한 분야에 범용적으로 적용 가능성을 보여주는 사례.

Ch2. Code Optimization Example — 단계별 최적화 실전

2.1 RISC CPU 개요 및 머신 코드 생성

RISC CPU 특성: - Uniform and orthogonal instruction set (단일 워드 명령어, opcode 위치 일정) - Identical general-purpose registers (동일한 범용 레지스터, 균일 사용) - Load/store-based architecture with simple addressing modes - → 쉬운 코드 생성과 최적화를 허용

RISC 명령어 형태:

Load:	load GP+offset, target-reg	(전역 변수 로드)
	load SP+offset, target-reg	(지역 변수 로드)
	loadi constant, target-reg	(상수 로드)
Compute:	add src-reg1, src-reg2, tgt-reg	(레지스터 연산)
	ldo src-reg, constant, tgt-reg	(레지스터 + 상수)
Store:	store src-reg, GP+offset	(전역 변수 저장)
	store src-reg, SP+offset	(지역 변수 저장)

스택 머신 코드 → RISC 코드 변환:

스택의 각 슬롯에 의사 레지스터(pseudo register, s0, s1, ...) 부여.
push→load, assign→store, 스택 깊이 추적.

x = y (x: 전역, y: 지역)의 변환 예:

push_const Lglob+4	→ loadi GP+4, s1	(x의 주소)
push_reg FP	→ copy FP, s2	
push_const 1, add	→ loadi 1, s3; add s2,s3,s2	
fetch	→ load s2(0), s2	(y의 값)
assign	→ store s1(0), s2	(y를 x에 저장)

결과: 의사 레지스터를 사용하는 **low-level IR** (물리 레지스터 미할당). 스택 머신 코드는 high-level IR.

2.2 예제 프로그램

```

int a[25][25];
main() {
    int i;
    for (i=0; i<25; i++)
        a[i][0] = 0;
}

```

메모리 공간과 레지스터: - 가상 주소 공간: code, global data, stack, heap - 32개 정수 레지스터: r0=항상 0, r27=GP(전역 데이터 포인터), r30=SP(스택 포인터) - 200 이상: 의사 레지스터 (미할당)

최종 최적화 어셈블리 (미리 보기):

```

ADDIL LR'a-$global$,%r27,%r1      ; &a 상위 21비트
LDO   RR'a-$global$(%r1),%r31     ; &a 하위 11비트
LDI   -25,%r23                    ; 카운터 = -25
$00000003
ADDIB,< 1,%r23,$00000003           ; 카운터++, <0이면 루프
STWM  %r0,100(%r31)                ; 0→a[i][0], 포인터+=100

```

2.3 비최적화 코드 (Un-optimized Code)

```

STW    0,-40(30)                    ; 0 → i (스택에 저장)
LDW    -40(30),206                  ; i를 r206에 로드
LDI    25,212                       ; 25를 r212에 로드
IFNOT  206 < 212 GOTO $02           ; i>=25이면 루프 탈출
NOP
$03: LDW    -40(30),206              ; i를 r206에 로드 [중복 #1]
      ADDILG LR'a-$global$,27,213   ; &a 상위비트 (21bit + GP)
      LDO    RR'a-$global$(213),208 ; &a 하위비트 (11bit)
      MULTI  100,206,214             ; i × 100 (행크기: 25 int ×
4byte)
      ADD    208,214,215              ; &a + i×100 = &a[i][0]
      STWS   0,0(215)                ; a[i][0] = 0
      LDW    -40(30),206              ; i를 r206에 로드 [중복 #2]
      LDO    1(206),216               ; i + 1
      STW    216,-40(30)              ; i = i+1 (스택에 저장)
      LDW    -40(30),206              ; i를 r206에 로드 [중복 #3]
      LDI    25,212                  ; 25를 r212에 로드 [중복]
      IF     206 < 212 GOTO $03       ; i<25이면 루프 반복
      NOP
$02:

```

주요 관찰: - 의사 레지스터(>200) 사용 중, r0/r27/r30만 실제 할당 - 스택 영역 (r30 기반) vs 전역 데이터 영역(r27 기반) - ADDILG + LDO 쌍 = 32비트 주소 상수 획득 (SPARC의 sethi+or, MIPS의 lui+ori에 해당) - **MULTI 100**: a[25][25]에서 한 행 = 25 × 4byte = 100byte - LDW -40(30),206이 한 반복에 4번 등장 — 심각한 중복

2.4 단계 1: Basic Block Optimization

Basic Block (BB): 제어 흐름이 시작점에서 들어와 끝점에서 나가며, 중간에 분기가 없는 연속 명령어 시퀀스. BB 헤더 = 분기 타겟 또는 제어 합류점(join point).

BB 구축 방법: 명령어→다음 실행 명령어 그래프 구축 → BB 헤더 식별 → 두 헤더 사이의 명령어가 하나의 BB → BB들의 연결이 **CFG (Control Flow Graph)**.

예제의 CFG:

```

BB0 (초기화 + 첫 조건)
|   STW 0,-40(30)
|   LDW -40(30),206
|   LDI 25,212
|   IFNOT 206<212 GOTO $02
|
↓
      ↗ (루프백)
BB1 (루프 본체)
|   LDW -40(30),206
|   ADDILG ...
|   LDO ...
|   MULTI ...
|   ADD ...
|   STWS ...
|   LDW -40(30),206
|   LDO 1(206),216
|   STW 216,-40(30)
|   LDW -40(30),206
|   LDI 25,212
|   IF 206<212 GOTO $03
|
↓
BB2 (루프 종료 후)
  Exit Code ...

```

지역(Local) 최적화 기법 — BB 내부에서 제어 흐름 정보 없이 수행:

(a) Load-to-Copy Optimization: store 직후 같은 위치 load → copy로 교체

변환 전: STW 0,-40(30); LDW -40(30),206

변환 후: STW 0,-40(30); COPY 0,206 ← 메모리 접근 → 레지스터 복사

(b) Local CSE (Common Subexpression Elimination): BB 내 동일 식 중복 계산 제거

변환 전: LDW -40(30),206 ... LDW -40(30),206 (같은 BB 내, r206 미변경)

변환 후: LDW -40(30),206 ... (삭제)

동일 식이 동일 타겟 레지스터를 가지면 두 번째 이후 삭제.

(c) Constant Folding: 컴파일 타임 계산 가능한 식을 상수로 교체

변환 전: if (4 > 3)

변환 후: if (true)

(d) Dead Code Elimination: 결과가 사용되지 않는 명령어 삭제

변환 전: x = y+1; ... x = 10 (첫 x 정의는 덮어쓰여져 사용 안 됨)

변환 후: ; ... x = 10

(e) Copy Propagation / Constant Propagation: copy의 소스로 사용처 교체

변환 전: x = y; ... z = x+100

변환 후: x = y; ... z = y+100 (x 대신 y 사용 → x가 dead되면 삭제 가능)

핵심 포인트: 이런 최적화들은 “프로그래밍을 잘하면 불필요”해 보이지만, 컴파일러가 생성하는 코드에서는 변수 읽기, 배열 주소 계산 등에서 자연스럽게 발생한다. 예제에서 같은 메모리 로드와 계산이 반복되는 것이 그 증거.

지역 최적화 적용 후 코드:

```
STW    0, -40(30)
COPY   0, 206          ← load-copy (원래 LDW)
LDI     25, 212        ← constant folding (IFNOT 삭제)

$03: LDW    -40(30), 206
      ADDILG LR'a-$global$, 27, 213
      LDO     RR'a-$global$(213), 208
      MULTI   100, 206, 214
      ADD     208, 214, 215
      STWS    0, 0(215)
                                     ← CSE (LDW -40(30), 206 삭제)

      LDO     1(206), 216
      STW     216, -40(30)
      COPY    216, 206          ← load-copy (원래 LDW)
                                     ← CSE (LDI 25, 212 삭제, 위에서 이미
계산)
      IF      206 < 212 GOTO $03
      NOP

$02:
```

Extended Basic Block: 들어오는 분기 없이 나가는 분기만 있는 연속 BB 체인. 동일한 지역 최적화를 확장 적용 가능.

2.5 단계 2: Global CSE (전역 공통 부분식 제거)

Available Expression: 지점 p에 도달하는 모든 경로에서 식 $x+y$ 가 계산되었고, 마지막 계산 이후 x나 y에 대한 재대입이 없으면 p에서 $x+y$ 가 “available”하다. p에서 $x+y$ 를 다시 계산하면 중복 → 제거.

가정: Value Numbering — 코드 생성기가 동일 계산(RHS)에 동일 타겟의 사 레지스터를 부여. 예: LDW -40(30), 206은 항상 r206에, LDI 25, 212는 항상 r212에.

적용 결과: BB0에서 이미 계산한 값이 BB1 시작점에서 available → BB1 내 중복 삭제:

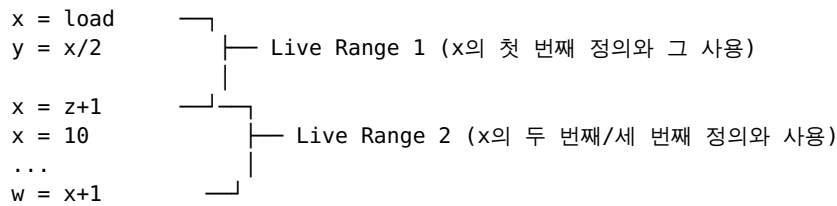
```
STW    0, -40(30)
COPY   0, 206
LDI     25, 212

$03:                                     ← available: r206(값), r212(25)
      ADDILG LR'a-$global$, 27, 213 ← LDW -40(30), 206 삭제됨
      LDO     RR'a-$global$(213), 208
      MULTI   100, 206, 214
      ADD     208, 214, 215
      STWS    0, 0(215)
      LDO     1(206), 216
      STW     216, -40(30)
      COPY    216, 206
      IF      206 < 212 GOTO $03      ← LDI 25, 212 삭제됨
      NOP
```

2.6 Definition, Use, Live Range

Definition (정의) = 쓰기(write). **Use** (사용) = 읽기(read). - $x = y + z$ 에서: 1개 정의(x), 2개 사용(y, z)

Live Range = 서로 “도달(reach)”할 수 있는 정의와 사용의 집합. 같은 저장 위치(레지스터 또는 메모리)에 접근.



두 종류: - **Register live range (web)**: 레지스터 할당의 단위 - **Memory live range**: 같은 메모리 위치에 접근

2.7 단계 3: Register Promotion (레지스터 승격)

목적: 메모리 연산(load/store)을 레지스터 연산(copy)으로 승격.

절차: 1. Memory Live Range 구축 = 같은 메모리 위치에 접근하는 store/load 집합 2. 주소 유도(address derivation) 계산: $STW\ 0, -40(30) \rightarrow 주소 = -40 + SP(r30)$ 3. 조건 확인: **singleton** 변수여야 함 (배열, 포인터, 구조체 불가) 4. 변환: **store** $\rightarrow$ **copy**, **load** $\rightarrow$ 삭제

프론트엔드가 같은 위치 접근 정보를 제공하면 좋고, 아니면 백엔드가 코드 분석으로 찾아야 함.

예제 적용: 변수 i에 대한 2개의 Memory Live Range:

MLR1: $STW\ 0, -40(30) \rightarrow COPY\ 0, 206$ (store $\rightarrow$ copy)
 $COPY\ 0, 206 \rightarrow$ (이미 copy) (load 이미 삭제)
 MLR2: $STW\ 216, -40(30) \rightarrow COPY\ 216, 206$ (store $\rightarrow$ copy)
 $COPY\ 216, 206 \rightarrow$ (이미 copy) (load 이미 삭제)

Register Promotion 후 코드:

```

COPY    0, 206                ; i = 0 (레지스터에!)
LDI     25, 212
NOP
$03: ADDILG LR'a-$global$, 27, 213
      LD0    RR'a-$global$(213), 208
      MULTI  100, 206, 214
      ADD    208, 214, 215
      STWS   0, 0(215)        ; a[i][0] = 0 (이것은 배열이므로 승격
불가)
$01: LD0    1(206), 216        ; i + 1
      COPY   216, 206          ; i = i + 1
      IF     206 < 212 GOTO $03
      NOP
  
```

$\rightarrow$ 변수 i에 대한 모든 **STW**, **LDW**가 사라졌다! 배열 a에 대한 STWS는 singleton이 아니므로 유지.

2.8 단계 4: Loop Optimization

세 가지 전통적 루프 최적화:

(a) LICM (Loop Invariant Code Motion): 루프 안에서 매 반복 같은 결과인 코드를 루프 밖(헤더)으로 이동

대상: $ADDILG\ LR'a-$global$, 27, 213 + LD0\ RR'a-$global$(213), 208$ — 배열 a의 기본 주소는 매 반복 동일 $\rightarrow$ 루프 전에 한 번만 계산.

(b) Strength Reduction (강도 감소): 비싼 연산(곱셈)을 싼 연산(덧셈)으로 교체

유도 변수 i 가 0,1,2,...,24로 변할 때:

```
원래: MULTI 100, i, offset      ; offset = i × 100 (곱셈, 매 반복)
변환: 새 변수 r218 도입
      초기: LDI 0,218           ; r218 = 0
      루프: LDO 100(218),220     ; r218 + 100 (덧셈, 매 반복)
            COPY 220,218        ; r218 = r218 + 100
```

(c) Induction Variable Elimination (유도 변수 제거): 강도 감소 후 원래 유도 변수가 루프 조건에만 쓰이면, 새 변수로 조건도 교체하여 원래 변수 제거

```
원래: IF 206 < 212 GOTO $03      ; i < 25
변환: LDI 2500,219               ; 25 × 100 = 2500
      IF 218 < 219 GOTO $03      ; offset < 2500
```

→ 원래 유도 변수 i (r206)와 상수 25(r212)가 불필요해짐.

루프 최적화 후 코드:

```
COPY 0,206                ; (나중에 dead code)
LDI 25,212                ; (나중에 dead code)
ADDILG LR'a-$global$,27,213 ← LICM: 루프 밖으로
LDO RR'a-$global$(213),208 ← LICM: 루프 밖으로
LDI 0,218                 ← 새 유도변수 초기화
LDI 2500,219              ← 새 상한값
$03:
COPY 218,214              ← MULTI 100,206,214 대체
ADD 208,214,215
STWS 0,0(215)
LDO 100(218),220          ← LDO 1(206),216 대체 (덧셈)
COPY 220,218              ← COPY 216,206 대체
IF 218 < 219 GOTO $03     ← IF 206 < 212 대체
```

2.9 단계 5: Building Register Live Ranges (Webs) + Dead Code Elimination

Register Live Range(Web) = 레지스터 정의와 사용의 도달 가능 집합. 레지스터 할당의 단위. 또한 Dead Code를 식별하는 데 도움 — 어떤 사용에도 도달하지 않는 정의는 dead.

Dead instructions 발견: - COPY 0,206 — r206은 이제 루프 안에서 사용되지 않음 (유도 변수가 r218로 교체됨) - LDI 25,212 — r212도 더 이상 사용되지 않음

이 두 명령어 삭제. 의사 레지스터 번호를 web 번호로 교체:

```
ADDILG LR'a-$global$,27,66
LDO RR'a-$global$(66),65
LDI 0,69
LDI 2500,71
$03:
COPY 69,67
ADD 65,67,68
STWS 0,0(68)
LDO 100(69),70
COPY 70,69
IF 69 < 71 GOTO $03
```

2.10 단계 6: Instruction Scheduling (명령어 스케줄링)

가정: 머신에 ALU 2개 → 한 클럭에 2개 명령어 동시 실행 가능.

원리: 서로 독립적인(데이터 의존성 없는) 명령어를 같은 사이클에 배치. 의존성이 있는 명령어 사이에 독립 명령어를 끼워넣음.

적용:

스케줄 전:	스케줄 후:	
COPY 69,67	COPY 69,67	
ADD 65,67,68	LDO 100(69),70	← code motion (독립적)
STWS 0,0(68)	ADD 65,67,68	
LDO 100(69),70	COPY 70,69	← code motion
COPY 70,69	STWS 0,0(68)	
IF 69<71 ...	IF 69<71 ...	

LDO 100(69),70은 r69만 읽으므로, ADD가 쓰는 r67, r68과 무관 → 앞으로 이동 가능.

2.11 단계 7: Register Allocation + Copy Elimination

Graph Coloring 기반 레지스터 할당: 1. **Interference Graph** 구축: 노드 = live range(web), 간선=동시에 살아있는 쌍 2. 물리 레지스터 수만큼의 색으로 그래프 컬러링 3. NP-complete → 휴리스틱 사용 4. 레지스터 부족 시 **Spilling**: 일부 web을 스택 메모리로 (def 직후 store, use 직전 load 추가)

두 **web**이 간섭하는 예: $x = \text{RHS1}; y = \text{RHS2}; z = x*10; \dots = y/x \rightarrow x$ 와 y 가 동시에 live → 같은 레지스터 불가

또한 **caller-save / callee-save** 레지스터 코드 생성 필요.

Copy Elimination — 두 가지 접근:

(a) **Copy Propagation**: copy를 dead code로 만듦

$y=x; z=y+1 \rightarrow z=x+1$ (y 대신 x 사용 → $y=x$ 가 dead되면 삭제)

(b) **Copy Coalescing**: copy의 소스와 타겟 live range가 간섭하지 않으면 두 web을 합쳐 같은 레지스터 할당 → COPY rN, rN 형태가 되어 삭제 - 부작용: 합친 노드의 간선이 합집합(union of edges) → 컬러링 어려워짐

예제 적용: - COPY 69,67: 67의 모든 사용을 69로 교체 → copy propagation → 삭제 - COPY 70,69: web 70과 69가 간섭 안 함 → coalescing → COPY 69,69 → 삭제

최종 코드 (non-scheduled version):

```
ADDILG LR'a-$global$,27,66
LDO    RR'a-$global$(66),65
LDI    0,69
LDI    2500,71
$03:
ADD    65,69,68          ; &a + offset
STWS   0,0(68)           ; a[i][0] = 0
LDO    100(69),69        ; offset += 100
IF     69 < 71 GOTO $03   ; offset < 2500?
```

→ 루프 본체 4개 명령어. 원래 13개에서 극적 축소.

참고: scheduled version에서는 coalescing이 불가능할 수 있다. 또한 compound instructions를 생성하면 더 축소 가능.

2.12 SSA Form (Static Single Assignment)

정의: 각 변수가 프로그램 전체에서 단 한 번만 정의되는 표현 형태. live range renaming과 유사.

현대 컴파일러의 표준 표현: Normal form $\rightarrow$ SSA form $\rightarrow$ 분석/최적화 $\rightarrow$ Normal form

일반 형태:	SSA 형태:
$a = x + y$	$a0 = x + y$
$b = a - 1$	$b0 = a0 - 1$
$a = y + b$	$a1 = y + b0$ $\leftarrow$ 다른 이름
$b = x * 4$	$b1 = x * 4$ $\leftarrow$ 다른 이름
$a = a + b$	$a2 = a1 + b1$

ϕ -function (파이 함수): 제어 합류점(join point)에 삽입. 어느 경로로 왔느냐에 따라 값을 선택. 실제로 실행되지 않는 표기법(notational fiction) — 분석과 최적화를 위한 것.

일반 형태:	SSA 형태:
$b = M[x]$	$b1 = M[x0]$
$a = 0$	$a1 = 0$
if $b < 4$	if $b1 < 4$
$a = b$	$a2 = b1$
	$a3 = \phi(a2, a1)$ $\leftarrow$ 합류점
$c = a + b$	$c1 = a3 + b1$

SSA의 이점 — 상수 전파(Constant Propagation) 예:

일반 형태:	SSA 형태:
$x = 1$	$x0 = 1$
$y = 2$	$y = 2$
$z = x + y$ $\leftarrow$ 어떤 x?	$z = x0 + y$ $\leftarrow x0=1$ 확실!
if (...)	if (...)
$h = x * 2$ $\leftarrow$ 어떤 x?	$h = x0 * 2$ $\leftarrow x0=1$ 확실!
$x = f(...)$	$x1 = f(...)$
$k = x - y$ $\leftarrow$ 어떤 x?	$x2 = \phi(x0, x1)$
	$k = x2 - y$ $\leftarrow x2$ 는 ϕ , 상수 아님

일반 형태에서는 각 x 사용이 어느 정의에서 왔는지 별도 분석 필요. SSA에서는 변수 이름만 보면 바로 확인 가능.

Ch3. Data Flow Analysis — 데이터 흐름 분석

3.1 데이터 흐름 분석이란?

주어진 함수가 데이터를 어떻게 다루는지에 대한 전역적 정보(global information)를 계산하고, 각 명령어 경계(instruction boundary)마다 그 정보를 제공하는 것.

용도 (최적화마다 다른 정보 필요): - Global CSE $\rightarrow$ **Available Expressions** (사용 가능한 표현식) - Register Live Range 구축 $\rightarrow$ **Reaching Definitions** (도달 정의) - Code Motion $\rightarrow$ **Live Variables** (생존 변수) - Constant Folding $\rightarrow$ 어떤 변수가 상수인지, 그 값이 무엇인지

핵심 원칙: 잘못된 정보 → 잘못된 최적화. 정확하지 않으면 보수적 근사
(**conservative approximation**) 필요. 최적화 기회를 놓치는 건 OK, 프로그램 의미를 바꾸면 절대 안 됨.

3.2 분석 전략: 2단계

1단계: Global Analysis – CFG 위에서 BB 단위, 각 BB 경계의 정보 계산
BB의 효과를 하나의 명령어처럼 요약하여 사용

2단계: Local Analysis – BB 내부, 각 명령어 경계의 정보 계산
전역 분석 결과(BB 경계 정보)를 출발점으로, 명령어 효과를 순차 적용

Local Analysis 예시 (Reaching Definitions):

BB 시작:		도달 정의 = {d1, d2} (전역 분석에서 제공)
x = 1 (d3)	→ kill d1	도달 정의 = {d2, d3}
y = 1 (d4)	→ kill d2	도달 정의 = {d3, d4}
z = x+y (d5)		도달 정의 = {d3, d4, d5}

이 BB의 요약: KILL={d1,d2}, GEN={d3,d4} (d5는 z의 마지막 정의이므로 GEN에 포함되어야 하지만 d3,d4가 x,y의 마지막 정의)

3.3 명령어의 효과 (Effect of an Instruction)

a = b + c의 경우: - **Uses:** 변수 b와 c - **Kills:** a에 대한 이전 정의 (a를 새로 정의하므로) - **Generates:** a에 대한 새 정의

이 효과를 명령어에 적용하여 데이터 흐름 정보를 “통과(thru)” 전파.

3.4 기본 블록의 효과 (Effect of a BB)

BB 내 명령어 효과를 합성(compose) → BB 전체의 요약.

핵심 3개념:

(1) Locally Exposed Use: BB 내에서 변수를 사용하는데, 그 사용 전에 같은 변수의 정의가 BB 내에 없는 경우. 즉 BB 외부에서 들어온 값을 사용.

(2) Kill: BB 내에서 변수를 정의하면, 그 변수에 대한 BB에 도달하는 모든 이전 정의를 kill.

(3) Locally Generated Definition: BB 내에서 변수에 대한 마지막 정의.

구체 예:

```
r4 = r1 + r2      ; r1 사용(exposed), r2 사용(exposed). r4 정의.
r2 = r4 + 1       ; r4 사용(NOT exposed, 위에서 정의됨). r2 재정의.
r5 = r3 + r1       ; r3 사용(exposed), r1 사용(exposed). r5 정의.
r1 = r5           ; r5 사용(NOT exposed). r1 재정의.
r4 = r2 * r4       ; r2, r4 사용(NOT exposed). r4 재정의.
r2 = r5           ; r5 사용(NOT exposed). r2 재정의.
if r2 > 100 goto L1 ; r2 사용(NOT exposed).
```

- **Locally exposed uses:** {r1, r2, r3} — BB 진입 시 값이 필요
- **Locally generated defs:** r1(=r5), r4(=r2*r4), r2(=r5) — 각 변수의 마지막 정의
- **Kill:** r4, r2, r1에 대한 이전(BB 외부) 정의들

3.5 정적 프로그램 vs 동적 실행 (Static vs Dynamic)

- 정적: 유한한 프로그램 코드
- 동적: 잠재적으로 무한한 실행 경로 (루프 때문)

데이터 흐름 분석의 정의: CFG의 각 정적 지점에, 그 지점에 도달하는(또는 출발하는) 모든 동적 실행 경로에 대해 참인 정보를 연관짓는 것.

방법: ① 각 경로의 BB 효과를 합성 → ② 진입점(또는 출구)의 정보에 적용 → ③ 모든 경로의 결과를 합쳐(merge) → 해당 지점의 정보.

루프가 있으면 경로가 무한 → 반복(iteration)으로 고정점(fixed point)까지 수렴.

3.6 Reaching Definitions (도달 정의) 분석

정의

- **Definition:** 변수 x에 값을 대입하는(또는 대입할 수 있는, e.g. 조건 실행) 명령어.
- **Reach:** 정의 d가 지점 p에 도달한다 = d 직후부터 p까지의 경로가 존재하고(3), 그 경로에서 d가 **kill**되지 **않음**.

```

      d1: a = 10
      d2: b = 11
      if e
        /      \
d3: a = 1      d5: c = a      ← a의 값은 d1에서 옴
d4: b = 2      d6: a = 4
        \      /
        합류점      ← a의 reaching defs = {d3, d6} (d1은 양쪽 모두
kill)
                                b의 reaching defs = {d4, d2} (d2는 왼쪽에서
kill, 오른쪽에서 살아남)

```

문제 정의

각 BB b에 대해: - **IN[b]**: b의 시작점에 도달하는 정의들의 집합 - **OUT[b]**: b의 끝점에서 나가는 정의들의 집합 - 표현: 비트 벡터 (함수 내 정의 하나당 1비트)

전달 함수 (Transfer Function) — Forward

IN[b]가 주어졌을 때 **OUT[b]**를 계산하는 것이 자연스럽다 (forward problem).

왜? 만약 OUT[b]에서 IN[b]를 역으로 구하려면, OUT[b]에 포함된 정의 d가 b 내부에서 정의된 것인 경우 IN[b]에 포함해야 하는지 불분명. → forward가 자연스러움.

$$OUT[b] = GEN[b] \cup (IN[b] - KILL[b])$$

- **GEN[b]**: BB b에서 새로 생성된 정의 (locally generated definitions)
- **KILL[b]**: 함수 내에서 b의 정의들과 같은 변수를 정의하는 다른 명령어들의 집합
- **IN[b] - KILL[b]**: b에 들어온 정의 중, b에서 kill되지 않고 살아남은 것

KILL의 정확한 의미 (주의!)

KILL[b]는 BB b의 정적 속성(**static property**)이다. “만약 이 정의가 b에 도달한다면 b를 통과하면서 무효화된다”는 **잠재적 능력**을 기술. 실제로 kill이 발생하려면 해당 정의가 IN[b]에 포함되어 있어야 한다.

예: B1이 함수 시작 $\rightarrow$ $IN[B1] = \{\}$ $\rightarrow$ KILL[B1]에 무엇이 있든 $\{\}$ - KILL = $\{\}$ $\rightarrow$ 효과 없음.

KILL은 “미래 정의를 죽인다”가 아니라 “같은 변수의 다른 정의 목록”이라는 정적 정보.

합류 연산 (Meet Operator) — Edges

단일 predecessor: $IN[b] = OUT[p]$ (그대로 전파)

Join node (여러 predecessor): 어떤 경로든 하나라도 도달 가능하면 포함 $\rightarrow$ **합집합(union)**

$IN[b] = OUT[p1] \cup OUT[p2] \cup \dots \cup OUT[pn]$

Cyclic Graph (루프)에서의 처리

방정식은 동일: $OUT[b] = GEN[b] \cup (IN[b] - KILL[b])$, $IN[b] = \cup OUT[pred]$

해: 방정식을 만족하는 **고정점(fixed point)** 해. 반복 계산으로 구한다.

Worklist Algorithm

```
/* Initialize */
OUT[Entry] = {}
for all nodes i: OUT[i] = {}
ChangeNodes = N (모든 노드)

/* Iterate */
while ChangeNodes != {} {
    remove i from ChangeNodes
    IN[i] =  $\cup OUT[p]$ , for all predecessors p of i
    oldout = OUT[i]
    OUT[i] =  $GEN[i] \cup (IN[i] - KILL[i])$ 
    if (oldout != OUT[i]) {
        for all successors s of i:
            add s to ChangeNodes
    }
}
```

예시 문제 풀이 — Reaching Definitions

CFG:

```
entry
  ↓
B1: d1: i=m-1,  d2: j=n,    d3: a=u1
  ↓
B2: d4: i=i+1,  d5: j=j-1    ← B3에서 루프백
  ↓
B3: d6: a=u2    B4: d7: i=u3
    (→B2 루프백)   ↓
                  exit
```

Step 1: GEN/KILL 계산

GEN[b] = BB 내 각 변수의 마지막 정의 모음 KILL[b] = BB 내 정의된 각 변수에 대해, 함수 전체에서 같은 변수를 정의하는 다른 명령어들

BB	정의	GEN	KILL (설명)
B1	d1(i), d2(j), d3(a)	{1,2,3}	{4,5,6,7} — i→{d4,d7}, j→{d5}, a→{d6}
B2	d4(i), d5(j)	{4,5}	{1,2,7} — i→{d1,d7}, j→{d2}
B3	d6(a)	{6}	{3} — a→{d3}
B4	d7(i)	{7}	{1,4} — i→{d1,d4}

Step 2: 반복 계산

초기: 모든 OUT = {}

1회차:

B1: IN = OUT[entry] = {}
OUT = {1,2,3} ∪ ({} - {4,5,6,7}) = {1,2,3}

B2: IN = OUT[B1] ∪ OUT[B3] = {1,2,3} ∪ {} = {1,2,3}
OUT = {4,5} ∪ ({1,2,3} - {1,2,7}) = {4,5} ∪ {3} = {3,4,5}

B3: IN = OUT[B2] = {3,4,5}
OUT = {6} ∪ ({3,4,5} - {3}) = {6} ∪ {4,5} = {4,5,6}

B4: IN = OUT[B2] = {3,4,5}
OUT = {7} ∪ ({3,4,5} - {1,4}) = {7} ∪ {3,5} = {3,5,7}

2회차 (B2의 predecessor B3의 OUT이 변화했으므로):

B2: IN = OUT[B1] ∪ OUT[B3] = {1,2,3} ∪ {4,5,6} = {1,2,3,4,5,6} ← 변화!
OUT = {4,5} ∪ ({1,2,3,4,5,6} - {1,2,7}) = {4,5} ∪ {3,4,5,6} = {3,4,5,6} ← 변화!

B3: IN = OUT[B2] = {3,4,5,6}
OUT = {6} ∪ ({3,4,5,6} - {3}) = {6} ∪ {4,5,6} = {4,5,6} ← 변화 없음

B4: IN = OUT[B2] = {3,4,5,6}
OUT = {7} ∪ ({3,4,5,6} - {1,4}) = {7} ∪ {3,5,6} = {3,5,6,7} ← 변화!

3회차: B2 재확인 → IN = {1,2,3} ∪ {4,5,6} = {1,2,3,4,5,6} → OUT = {3,4,5,6} → 변화 없음 → 수렴!

최종 결과:

	IN	OUT
entry:	{}	{}
B1:	{}	{1,2,3}
B2:	{1,2,3,4,5,6}	{3,4,5,6}
B3:	{3,4,5,6}	{4,5,6}
B4:	{3,4,5,6}	{3,5,6,7}

해석: B2 시작점에서 IN={1,2,3,4,5,6} — d1~d6 모두 도달 가능. B1에서 직접(d1,d2,d3) + B3 루프백(d4,d5,d6). d7은 B2에 도달하지 않음(B4는 B2의 predecessor가 아님).

3.7 Live Variables (생존 변수) 분석

정의

- **Live:** 변수 v 가 지점 p 에서 $\text{live} = p$ 에서 시작하는 어떤 경로를 따라 v 의 값이 (재정의 전에) 사용됨
- **Dead:** p 이후 어떤 경로에서도 v 가 (재정의 전에) 사용되지 않음

용도: 레지스터 할당 (dead 변수의 레지스터를 다른 용도로 사용 가능), Dead Code Elimination.

전달 함수 — **Backward**

OUT[b]가 주어졌을 때 IN[b]를 계산 (**backward problem**):

$$\text{IN}[b] = \text{USE}[b] \cup (\text{OUT}[b] - \text{DEF}[b])$$

- **USE[b]:** BB b 의 locally exposed uses (사용 전에 BB 내 정의가 없는 변수)
- **DEF[b]:** BB b 에서 정의된 변수들의 집합
- **OUT[b] - DEF[b]:** b 이후에 live한 변수 중, b 에서 재정의되지 않은 것

합류 연산

Live Variables의 “합류 노드” = 여러 **successor**를 가진 노드 (분기점). 정보가 뒤에서 앞으로 흐르므로.

$$\text{OUT}[b] = \text{IN}[s_1] \cup \text{IN}[s_2] \cup \dots \cup \text{IN}[s_n] \quad (\text{모든 successor } s)$$

합집합(union): 어떤 후속 경로에서든 사용될 가능성이 있으면 live.

Worklist Algorithm (Backward)

```
/* Initialize */
IN[Exit] = {}
for all nodes i: IN[i] = {}
ChangeNodes = N

/* Iterate */
while ChangeNodes != {} {
    remove i from ChangeNodes
    OUT[i] =  $\cup$  IN[s], for all successors s of i
    oldin = IN[i]
    IN[i] = USE[i]  $\cup$  (OUT[i] - DEF[i])
    if (oldin != IN[i]) {
        for all predecessors p of i:
            add p to ChangeNodes      ← RD와 반대 방향!
    }
}
```

3.8 Framework 대비표

항목	Reaching Definitions	Live Variables
Domain	Sets of Definitions	Sets of Variables
Transfer Function $f_b(x)$	$\text{GEN}[b] \cup (x - \text{KILL}[b])$	$\text{USE}[b] \cup (x - \text{DEF}[b])$

Direction	Forward: $OUT[b] = f_b(IN[b])$	Backward: $IN[b] = f_b(OUT[b])$
Generate	$GEN[b]$: BB 내 정의	$USE[b]$: BB 내 사용
Propagate	$IN[b] - KILL[b]$	$OUT[b] - DEF[b]$
Merge Operation	$IN[b] = \cup OUT[pred]$	$OUT[b] = \cup IN[succ]$
Initialization	$OUT[b] = \{\}$	$IN[b] = \{\}$
Boundary Condition	$OUT[entry] = \{\}$	$IN[exit] = \{\}$
변화 시 전파 대상	successor 추가	predecessor 추가

두 분석은 완벽하게 대칭적이며, 이 구조가 Ch4(Foundations of Dataflow Analysis)의 일반화된 프레임워크(격자 이론, 단조 함수)의 기초가 된다.

7-보충. Live Variables 상세 풀이 (같은 CFG 예제)

같은 CFG(B1~B4)에서 LV를 RD와 동일한 수준으로 상세하게 풀어본다.

USE/DEF 계산 — 한 줄씩 추적:

B1 분석:

d1: $i = m - 1$
 오른쪽(읽기): $m \rightarrow$ BB 시작부터 여기까지 m 을 쓴 적 없음 $\rightarrow$ 첫 등장이 읽기 $\rightarrow$ USE에 m 추가!
 왼쪽(쓰기): $i \rightarrow$ DEF에 i 추가. i 의 첫 등장이 쓰기 $\rightarrow$ USE에 i 안 넣음.
 d2: $j = n$
 오른쪽 $n \rightarrow$ 첫 등장 읽기 $\rightarrow$ USE에 n ! 왼쪽 $j \rightarrow$ DEF에 j .
 d3: $a = u1$
 오른쪽 $u1 \rightarrow$ 첫 등장 읽기 $\rightarrow$ USE에 $u1$! 왼쪽 $a \rightarrow$ DEF에 a .
 $\rightarrow USE = \{m, n, u1\}, DEF = \{i, j, a\}$

B2 분석 (핵심!):

d4: $i = i + 1$
 ★ 오른쪽 i (읽기)가 왼쪽 i (쓰기)보다 먼저 실행됨!
 CPU 동작: ① 오른쪽 i 값 읽음 $\rightarrow$ ② 1 더함 $\rightarrow$ ③ 왼쪽 i 에 씀
 i 의 첫 등장이 읽기 $\rightarrow$ USE에 i 추가! 왼쪽 $i \rightarrow$ DEF에 i .
 d5: $j = j - 1$
 j 의 첫 등장이 읽기 $\rightarrow$ USE에 j 추가! 왼쪽 $j \rightarrow$ DEF에 j .
 $\rightarrow USE = \{i, j\}, DEF = \{i, j\}$
 주의: USE와 DEF에 같은 변수 가능! "정의도 하지만 사용이 먼저"

B3: $a = u2 \rightarrow USE = \{u2\}, DEF = \{a\}$. B4: $i = u3 \rightarrow USE = \{u3\}, DEF = \{i\}$.

반복 계산 (Backward) — exit에서 entry쪽으로 계산. 초기: 모든 $IN = \{\}$:

1회차:

B4: $OUT = IN[exit] = \{\}$
 $IN = \{u3\} \cup (\{\} - \{i\}) = \{u3\}$
 해석: $u3$ 만 live. B4에서 $i=u3$ 할 때 $u3$ 를 읽어야 하니까. 변
 화!

B3: $OUT = IN[B2] = \{\}$ (B2의 IN 아직 $\{\}$)
 $IN = \{u2\} \cup (\{\} - \{a\}) = \{u2\}$
 해석: $u2$ 만 live.

변화!

B2: OUT = IN[B3] $\cup$ IN[B4] = {u2} $\cup$ {u3} = {u2,u3}
IN = {i,j} $\cup$ ({u2,u3} - {i,j}) = {i,j} $\cup$ {u2,u3} = {i,j,u2,u3}
해석: i,j는 USE(BB 내 사용). u2,u3는 DEF에 없어서 통과 전파. 변화!
화!

B1: OUT = IN[B2] = {i,j,u2,u3}
IN = {m,n,u1} $\cup$ ({i,j,u2,u3} - {i,j,a}) = {m,n,u1} $\cup$ {u2,u3} = {m,n,u1,u2,u3}
해석: i,j는 B1의 DEF에 있어서 제거! 새로 정의하므로 이전 값 불필요. 변화!
화!

2회차 (B2의 IN이 변했으므로 B3의 OUT 영향):

B3: OUT = IN[B2] = {i,j,u2,u3} ← 1회차에서는 {}였음!
IN = {u2} $\cup$ ({i,j,u2,u3} - {a}) = {u2} $\cup$ {i,j,u2,u3} = {i,j,u2,u3}
해석: i,j,u3는 B3에서 안 건드려서 OUT에서 통과. B2에서 i,j를, B4에서 u3를 쓸 것. 변화!

B2: OUT = IN[B3] $\cup$ IN[B4] = {i,j,u2,u3} $\cup$ {u3} = {i,j,u2,u3} ← 1회차와 동일!
IN = {i,j,u2,u3} 변화 없음
→ 수렴!

최종: B1:OUT={i,j,u2,u3},IN={m,n,u1,u2,u3} / B2:OUT=IN={i,j,u2,u3} / B3:OUT=IN={i,j,u2,u3} / B4:OUT={},IN={u3}

결과 해석: - B2의 i,j: B2 내 i=i+1, j=j-1에서 오른쪽에 사용 → live - B2의 u2,u3: B2가 안 건드려서 통과. B3(u2), B4(u3)에서 사용 예정 - B3의 i,j: B3 자체에서 사용 안 하지만 OUT에서 통과(DEF={a}에 없음). B2에서 쓸 예정 - B1의 IN에서 i,j 빠짐: B1에서 i=m-1, j=n으로 새로 정의 → 이전 값 불필요 - B4의 OUT={}: exit 후 아무것도 안 쓰이므로

Ch4. Foundations of Data Flow Analysis — 데이터 흐름 분석의 수학적 기초

1. 이 장이 답하는 4가지 질문

Ch3에서 RD와 LV의 반복 알고리즘을 배웠다. 그런데 자연스러운 의문이 생긴다:

① 수렴: 반복 계산이 반드시 끝나는가? 영원히 돌 수도 있지 않나? ② 정확성: 끝났을 때 그 답이 안전한가? 잘못된 최적화를 유발하지 않나? ③ 정밀도: 답이 얼마나 좋은가? 불필요하게 보수적이진 않나? ④ 속도: 얼마나 빨리 끝나는가? 노드 방문 순서가 영향을 주는가?

이 질문에 답하기 위해, 개별 분석(RD, LV 등)을 넘어서 모든 데이터 흐름 분석에 공통으로 적용되는 수학적 프레임워크를 세운다.

2. 통합 프레임워크 (V, A, F)

모든 데이터 흐름 문제는 딱 세 가지로 정의된다:

- **V** = 값의 도메인. 분석에서 다루는 정보의 “세계”. 예를 들어 RD에서는 “정의들의 부분집합들의 집합”이다. 정의가 d1, d2, d3이면 $V = \{\{\}, \{d1\}, \{d2\}, \{d3\}, \{d1,d2\}, \{d1,d3\}, \{d2,d3\}, \{d1,d2,d3\}\}$ — 총 $2^3 = 8$ 가지.
- **$\wedge$** = meet 연산자. 여러 경로의 정보를 합치는 방법. RD에서는 $\cup$ (union), Available Expressions에서는 $\cap$ (intersection).
- **F** = 전달 함수의 집합. 각 BB가 정보를 어떻게 변형하는가. RD에서는 $f(x) = \text{GEN} \cup (x - \text{KILL})$.

이렇게 (V, $\wedge$, F)로 정의하면, 같은 성질을 가진 문제들에 대해 수렴·정확성·정밀도를 한번에 증명할 수 있다. 개별 분석마다 따로 증명할 필요가 없다.

3. Meet 연산자의 4가지 성질

$\wedge$ 가 만족해야 하는 성질이 있다. $\cup$ (union)으로 예를 들면:

- **교환법칙**: $\{d1\} \cup \{d2\} = \{d2\} \cup \{d1\}$. 합치는 순서가 상관없다.
- **멱등성**: $\{d1\} \cup \{d1\} = \{d1\}$. 자기 자신과 합쳐도 변하지 않는다.
- **결합법칙**: $(\{d1\} \cup \{d2\}) \cup \{d3\} = \{d1\} \cup (\{d2\} \cup \{d3\})$. 괄호 위치가 상관없다.
- **Top 원소**: $\{d1\} \cup \{\} = \{d1\}$. 빈 집합($\top$)과 합치면 자기 자신. $\top$ 은 “아무 정보도 없는 초기 상태”.

이 4가지 성질로부터 **반순서($\leq$)**를 정의할 수 있다:

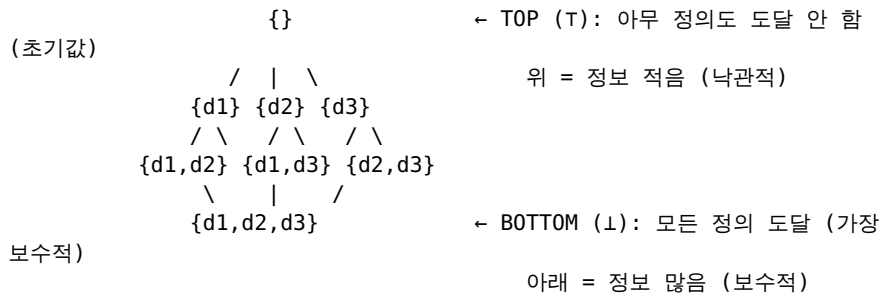
$$x \leq y \iff x \wedge y = x$$

무슨 뜻인가? “x와 y를 합쳤더니 x가 나왔다”는 것은 “x가 이미 y의 정보를 포함하고 있다”는 뜻이다. 즉 x가 y보다 더 많은 정보를 가지고 있어서, y를 합쳐봤자 변하지 않는다.

구체 예: $\wedge = \cup$ 일 때, $\{d1,d2\} \leq \{d1\}$ 인가? $\{d1,d2\} \cup \{d1\} = \{d1,d2\} =$ 왼쪽과 같음 $\rightarrow$ **YES!** $\{d1,d2\}$ 가 $\{d1\}$ 보다 “아래”(더 많은 정보, 더 보수적).

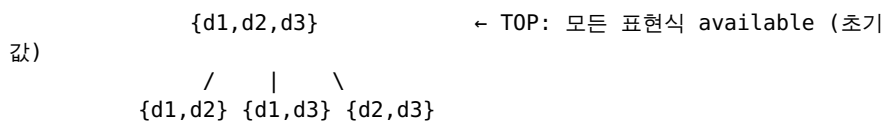
4. 격자 다이어그램 — 가능한 모든 값을 계층으로 그린 지도

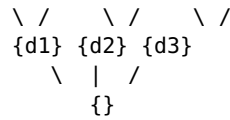
정의가 3개(d1, d2, d3)이고 $\wedge = \cup$ 인 RD의 격자:



반복 알고리즘은 **T(맨 위)**에서 시작해서 아래로만 내려간다. 위로 돌아가는 일은 없다!

$\wedge = \cap$ (Available Expressions)이면 방향이 반대:





← BOTTOM: 아무것도 available 아님

5. 격자의 높이 — 수렴의 핵심

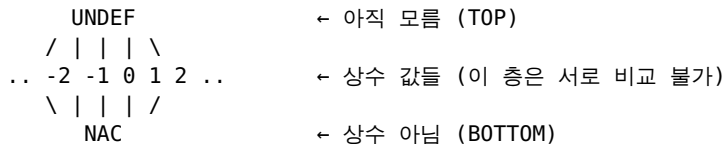
하강 체인: 격자에서 위에서 아래로 내려가는 수열. 예: $\{ \} \rightarrow \{d1\} \rightarrow \{d1, d2\} \rightarrow \{d1, d2, d3\}$

격자의 높이: 가장 긴 하강 체인의 길이. RD에서 정의 3개 $\rightarrow$ 높이 = 3.

왜 중요한가? 반복할 때마다 격자를 한 칸 이상 내려간다. 높이가 유한하면 영원히 내려갈 수 없다 $\rightarrow$ 반드시 멈춘다!

비유: 계단을 한 칸씩 내려가는데, 계단이 3칸이면 최대 3번만 내려가면 바닥.

상수 전파의 격자는 흥미롭다: 상수값이 무한개($-\infty, \dots, -1, 0, 1, \dots, \infty$)이지만, 하강 체인의 길이는 2뿐이다 (UNDEF $\rightarrow$ 상수 $\rightarrow$ NAC). 그래서 수렴이 보장된다.



높이 = 2: UNDEF에서 시작 $\rightarrow$ 상수 하나로 $\rightarrow$ NAC로 (최대 2단계)

6. 단조성 (Monotonicity) — “내려가면 계속 내려간다”

프레임워크가 단조(monotone)이다 $\iff x \leq y$ 이면 $f(x) \leq f(y)$

직관: “입력이 격자에서 내려가면, 출력도 내려간다.” 위로 뛰어오르는 일이 없다.

이것이 왜 수렴을 보장하는가?

RD ($\lambda=U$) 기준으로 설명:

초기: 모든 $OUT[b] = \{ \}$ (빈 집합, Top)

반복할수록 OUT 집합이 커진다:

$OUT_0 = \{ \} \rightarrow OUT_1 = \{d1\} \rightarrow OUT_2 = \{d1, d2\} \rightarrow \dots$
(정의가 추가되어 감)

왜 커지지만 하는가?

$IN[b] = U \cup OUT[pred] \rightarrow$ union이므로 빼는 건 없고 추가만 됨
 $OUT[b] = GEN \cup (IN - KILL) \rightarrow IN$ 이 커지면 OUT 도 커짐

격자 순서와 집합 포함의 관계 ($\lambda=U$):

$x \leq y \iff y \subseteq x$ (x가 y보다 큰 집합이면 x가 아래)
 $x \geq y \iff x \subseteq y$ (x가 y보다 작은 집합이면 x가 위)
★ 반직관적! 격자에서 $\geq$ (위)가 “더 작은 집합”을 뜻함!

$\{ \} \subseteq \{d1\} \subseteq \{d1, d2\}$ (집합: 커짐 $\rightarrow$)
 $\{ \} \geq \{d1\} \geq \{d1, d2\}$ (격자: 위에서 아래로)
Top(위, 작음) $\longrightarrow$ Bottom(아래, 큼)

왜 반드시 멈추는가?

비트벡터 크기가 유한 (정의 n 개 $\rightarrow$ 최대 n 비트)
 한 번 1이 된 비트는 0으로 돌아가지 않음 (union이니까)
 $\rightarrow$ 최대 n 번 반복하면 모든 비트가 확정됨 $\rightarrow$ 수렴!

단조성의 역할:

단조성 = "입력이 커지면 출력도 커진다" (또는 같다)
 $\rightarrow$ "비트가 한 번 1이 되면 다시 0이 안 된다"를 보장
 $\rightarrow$ 격자에서 "한 방향(아래)으로만 이동"을 보장

주의: 단조성은 $f(x) \leq x$ 를 뜻하지 않는다! 예를 들어 $GEN=\{d1\}$, $KILL=\{d2\}$, $x=\{d2\}$ 이면 $f(x)=\{d1\}$ 인데, $\{d1\}$ 과 $\{d2\}$ 는 격자에서 비교 불가능하다. 단조성은 "입력 간의 관계가 출력에서 보존된다"는 뜻이다.

동치 표현: $f(x \wedge y) \leq f(x) \wedge f(y)$. 이것의 의미가 매우 중요하다: - 왼쪽 $f(x \wedge y)$: 먼저 합치고 나서 함수 적용 $\rightarrow$ 이것이 반복 알고리즘이 하는 것! - 오른쪽 $f(x) \wedge f(y)$: 각각 함수 적용한 후 합치기 $\rightarrow$ 이것이 이상적 해가 하는 것! - $\leq$ 이므로: 반복 알고리즘의 결과가 이상적 해보다 아래(더 보수적) $\rightarrow$ 안전하지만 정밀도는 낮을 수 있다.

7. 분배성 (Distributivity) — 정밀도 손실이 없는 조건

프레임워크가 분배적(distributive)이다 $\iff f(x \wedge y) = f(x) \wedge f(y)$ (등호!)

단조성에서는 $\leq$ (이하)였는데, 분배성에서는 $=$ (등호)다!

"먼저 합치고 적용"과 "적용하고 합치기"가 완전히 같은 결과를 준다. 즉, 반복 알고리즘이 이상적 해만큼 정밀한 해를 준다!

RD와 LV는 분배적인가? 확인해보자:

RD: $f(x) = GEN \cup (x - KILL)$, $\wedge = \cup$

$$\begin{aligned} f(x \cup y) &= GEN \cup ((x \cup y) - KILL) \\ &= GEN \cup ((x - KILL) \cup (y - KILL)) \quad \leftarrow \text{집합론의 분배법칙!} \\ &= (GEN \cup (x - KILL)) \cup (GEN \cup (y - KILL)) \\ &= f(x) \cup f(y) \end{aligned}$$

$$f(x \wedge y) = f(x) \wedge f(y) \rightarrow \text{등호 성립!} \rightarrow \text{분배적!} \checkmark$$

RD, LV, Available Expressions 모두 분배적 $\rightarrow$ 반복 알고리즘이 최선의 해를 준다.

상수 전파는 분배적이지 않다. 구체적으로 왜?

$$\begin{array}{ccc} \text{경로 A: } x=2, y=3 & & \text{경로 B: } x=3, y=2 \\ & \searrow \quad \swarrow & \\ & \rightarrow z = x + y \leftarrow & \end{array}$$

"각각 적용 후 합치기" (이상적):

경로 A에서: $z = 2 + 3 = 5$

경로 B에서: $z = 3 + 2 = 5$

합치면: $z = 5$ (양쪽 다 5이므로 상수!)

"합치고 적용" (반복 알고리즘):

x 를 합침: 2와 3 $\rightarrow$ 다른 값이므로 NAC (Not A Constant)

y 를 합침: 3과 2 $\rightarrow$ 역시 NAC

$z = \text{NAC} + \text{NAC} = \text{NAC}$

결과: 이상적 해는 $z=5$ (상수)인데, 반복 알고리즘은 $z=\text{NAC}$ (상수 아님)

$\rightarrow$ 정밀도 손실! 하지만 "상수가 아닌 걸 상수로 잘못 판단"하지는 않으므로 안전.

8. 해의 계층: $FP \leq MFP \leq MOP \leq \text{Perfect Solution}$

“해”에도 등급이 있다:

- **Perfect Solution (이상적 해):** “이 프로그램이 실제로 실행될 때 가능한 경로만” 고려한 해. 가장 정밀하지만, 어떤 경로가 실행 가능한지 판별하는 것은 결정 불가능(undecidable)이므로 계산할 수 없다.
- **MOP (Meet Over all Paths):** CFG에 존재하는 모든 경로(실행 가능한 아니든)를 고려한 해. 실행 불가능한 경로도 포함하므로 Perfect보다 보수적.
- **MFP (Maximum Fixed Point):** 반복 알고리즘이 수렴한 결과. 데이터 흐름 방정식을 만족하는 모든 해 중 가장 큰(가장 덜 보수적인) 것.

이들의 관계:

$FP \leq MFP \leq MOP \leq \text{Perfect Solution}$

← 더 보수적(안전하지만 기회 놓침) ———— 더 정밀(이상적) →

분배적이면: $MFP = MOP$ (반복 알고리즘이 최선!)

단조적이지만 비분배면: $MFP < MOP$ (정밀도 손실, 하지만 안전)

9. 정확성 — 경로를 무시하면 왜 위험한가

RD에서 정상적인 경우:

d1: x= d2: x=
 \ /
 u1: =x
RD: {d1, d2} (안전)

경로를 무시한 경우:

d1: x= d2: x=
 \ (무시!)
 u1: =x
RD: {d1} (위험!)

격자에서 보면:

{d1, d2} = 아래쪽 (보수적, 안전)
{d1} = 위쪽 (위험! 정보 손실)

경로를 무시하면 격자에서 위로 올라간다. 이것은 “실제로 도달하는 정의를 놓치는 것”이므로 잘못된 최적화를 유발할 수 있다.

반복 알고리즘은 union으로 모든 경로를 합치므로 절대 경로를 무시하지 않는다
→ 항상 아래로만 이동 → 정확(correct).

10. 수렴 속도 — Reverse Post-Order

수렴 속도는 노드 방문 순서에 달려있다.

Forward 분석에서는 **reverse post-order (rPostOrder)**로 방문하면 가장 빠르다. 이것은 대략 “위에서 아래로”(위상 정렬에 가까운) 순서이다. 정보가 한 패스에 위에서 아래로 쭉 전파될 수 있다.

rPostOrder 계산법:

1단계: DFS(깊이 우선 탐색)로 그래프를 순회하면서,
각 노드에서 “빠져나올 때” 번호를 매긴다 (post-order).

Visit(n):

 for each unvisited successor s:

 Visit(s)

 PostOrder(n) = count++ ← 재귀가 끝날 때 번호 부여

2단계: post-order를 뒤집는다.

$rPostOrder(i) = \text{전체노드수} - PostOrder(i)$

루프가 없는 그래프에서는 rPostOrder로 한 번만 순회하면 수렴한다. 루프가 있으면 루프의 백엣지(back edge) 때문에 추가 반복이 필요한데, 실제 프로그램에서 평균 반복 횟수는 약 **2.75회**(루프 중첩 깊이에 비례).

Backward 분석에서는 rPostOrder의 역순으로 방문한다.

11. Ch4가 컴파일러의 어디에 연결되는가

Ch4의 수학적 개념들은 추상적으로 보이지만, 실제 컴파일러의 구체적인 부분들과 직접 연결된다.

각 데이터 흐름 분석이 어떤 최적화에 사용되는지: - **RD (분배적)** → Live Range 구축(→Register Allocation), Register Promotion, 유도변수 추적(→Strength Reduction) - **LV (분배적)** → Dead Code 제거, Register Allocation(live인 변수만 레지스터 필요), LICM 안전성 확인(이동 후 변수가 live인지) - **Available Expressions (분배적)** → Global CSE(available하면 중복 제거), LICM(루프 불변이면서 available하면 이동 가능) - **Constant Propagation (비분배!)** → 상수 접기/전파. 반복 알고리즘으로는 정밀도 한계 → 실제 GCC/LLVM에서는 SSA 기반의 더 정교한 알고리즘(Sparse Conditional Constant Propagation) 사용

Ch4의 각 개념이 왜 필요한지: - 단조성: “새로 만든 분석의 반복 알고리즘이 무한 루프에 빠지지 않는가?” 확인 - 분배성: “반복 알고리즘으로 충분한가, 더 정교한 알고리즘이 필요한가?” 판별 - 격자 높이: “100만 줄 코드에서 분석이 몇 회 반복으로 끝나는가?” 예측 - **rPostOrder**: GCC/LLVM이 실제로 BB를 방문하는 순서. 수렴을 가장 빠르게 만들

비유: Ch3이 “자동차 운전법”이라면 Ch4는 “엔진의 원리”. 운전만 할 거면 Ch3으로 충분하지만, 새 차를 설계하거나(새 분석 만들기), 고장 진단(왜 수렴 안 하는지), 성능 튜닝(더 빠른 수렴)을 하려면 Ch4가 필요하다.

Ch5. Global Common Subexpression Elimination — 이미 계산한 걸 또 계산하지 마라

1. 이 장이 하는 일

Ch2에서 맛보기로 봤던 Global CSE를 정식으로 다룬다. BB 내부 중복(Local CSE)은 Ch2에서 배웠는데, **BB 경계를 넘어서** 같은 식이 이미 계산되었으면 다시 계산하지 않고 이전 결과를 재사용하는 것이 Global CSE이다. 핵심 질문: “어떤 식이 특정 지점에서 이미 계산되어 있는가?” → 이것을 판별하는 데이터 흐름 분석이 **Available Expression Analysis**이다.

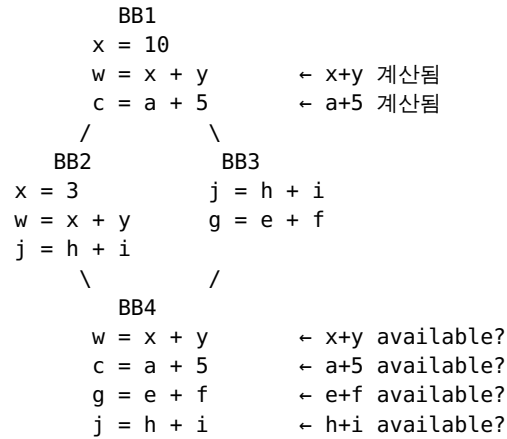
2. Available Expression — 정확한 정의

표현식 E가 지점 P에서 **available**하다 = P에 도달하는 모든 경로에서 ① E가 최소 한 번 계산되었고 ② 그 마지막 계산 이후 E의 피연산자가 재정의되지 않았다.

핵심: “모든 경로”! RD에서는 “어떤 경로든 하나라도”(∃, union)이었는데, AE에서는 “모든 경로에서”(∀, intersection)이다. 한 경로에서만 계산되고 다른 경로에서는 안 되었다면, 런타임에 그 다른 경로로 올 수 있으므로 이전 결과를 재사용하면 위험하다.

중요한 관찰: E의 값이 경로마다 다를 수 있다! Available은 “값이 같다”는 뜻이 아니라 “이미 계산되어 결과가 레지스터에 있다”는 뜻이다.

슬라이드 예제 CFG로 구체적으로 확인:



BB4 시작점에서 각 표현식을 판별:

- **x+y**: BB2 경로에서 x=3으로 x가 재정의된 후 w=x+y로 다시 계산됨 → available. BB3 경로에서 BB1에서 계산되었고 BB3에서 x나 y를 안 건드림 → available. 모든 경로에서 available → **YES!**
- **a+5**: BB2 경로에서 BB1에서 계산, BB2에서 a를 안 건드림 → available. BB3 경로도 동일 → **YES!**
- **e+f**: BB2 경로에서 계산 안 됨 → **NOT available!** BB3에서는 계산되지만 “모든 경로” 불충족.
- **h+i**: BB2에서 계산됨, BB3에서도 계산됨 → 모든 경로 → **YES!**

3. AE의 데이터 흐름 프레임워크

도메인: 비트 벡터 (프로그램 내 고유 표현식 하나당 1비트). 방향: Forward. $\wedge = \cap$ (intersection).

전달 함수: $OUT[b] = GEN[b] \cup (IN[b] - KILL[b])$ — RD와 같은 형태이지만 GEN/KILL의 의미가 다르다.

경계 조건과 초기화의 구분:

- 경계 조건(Boundary): $IN[entry] = \{\}$ — 함수 진입 시 아무것도 available 아님. 이것은 **사실(fact)**이며 안 바뀐다. (dummy entry 모델 사용 시 $OUT[entry] = \{\}$ 로 표기하기도 함.)
- 초기화(Initialization): $OUT[b] = U$ (전체 집합) — 나머지 BB의 임시값. $\wedge = \cap$ 이므로 $Top =$ 전체 집합. **$\{\}$ 로 초기화하면 $IN[b] = \cap OUT[pred]$ 에서 하나라도 $\{\}$ 이면 결과가 $\{\}$ → 모든 게 사라져서 쓸모없는 결과가 된다!**

RD와의 핵심 차이: RD($\wedge = U$)는 $Top = \{\}$, 초기화 $OUT[b] = \{\}$. AE($\wedge = \cap$)는 $Top = U$, 초기화 $OUT[b] = U$. 반복할수록 RD는 집합이 커지고, AE는 intersection으로 깎여 작아진다.

4. 명령어의 GEN/KILL — AE용

명령어 $w = x + y$ 의 효과:

- GEN: $\{x+y\}$ — 이 표현식이 이제 available
- KILL: w를 피연산자로 포함하는 모든 표현식 — w의 값이 바뀌었으니 이전

에 w를 쓰던 식은 무효

RD에서 KILL은 “같은 변수의 다른 정의들”이었지만, AE에서 KILL은 “타겟 변수를 피연산자로 포함하는 모든 표현식”이다. 관점이 다르다: RD는 정의 단위, AE는 표현식 단위.

특수 케이스 $x = x + y$:

- ① 오른쪽에서 $x + y$ 를 계산 → GEN: $\{x+y\}$
- ② 결과를 x에 대입 → KILL: x를 포함하는 표현식 ($a+x$, $x+y$ 등)
- ③ 방금 GEN한 $x+y$ 도 x를 포함! → GEN되자마자 KILL!
→ $x+y$ 는 available하지 않음!

5. BB 전달 함수 합성

BB 안에 명령어 a, b가 순서대로 있을 때:

```
out1 = gen1 u (in1 - kill1)
out2 = gen2 u (out1 - kill2)
      = gen2 u ((gen1 u (in1 - kill1)) - kill2)
      = gen2 u (gen1 - kill2) u (in1 - (kill1 u kill2))
```

BB 전체: GEN = $gen2 \cup (gen1 - kill2)$
KILL = $kill1 \cup kill2$

직관: KILL은 합집합(어떤 명령어든 kill한 건 BB 전체의 kill). GEN은 “뒤 명령어가 앞 GEN을 kill할 수 있음”(gen1 - kill2). 뒤가 이김.

6. CSE 제거 절차 — 2단계

Step 1: Available Expression Analysis(전역) — 반복 알고리즘으로 각 BB 시작점의 available 표현식 비트벡터를 계산.

Step 2: Value Numbering(BB 내부) — available 표현식으로 값 테이블을 초기화하고, BB 내에서 중복 표현식 발견 시 이전 결과로 교체.

Ch2 예제와 연결: BB0에서 $r212=25$ 를 계산. BB1 시작에서 $r212$ 가 available → BB1 내 LDI 25,212 삭제 가능.

7. PRE (Partial Redundancy Elimination) — 부분 중복 제거

Global CSE는 “완전히 중복인(fully redundant)” 계산만 제거한다. 하지만 “부분적으로 중복인(partially redundant)” 경우도 있다:

변환 전:	변환 후:	
if (a > 0)	t = b + c	← 새로 삽입!
d = b + c	if (a > 0)	
e = b + c	d = t	← 이전 결과 재사용
	e = t	← 이전 결과 재사용

$b + c$ 는 $a > 0$ 일 때만 두 번 계산(부분적 중복). if 앞에 $t = b + c$ 를 미리 삽입하면, 이후의 $b + c$ 가 모두 완전히 중복이 되어 Global CSE로 제거 가능.

하지만 아무 데나 삽입하면 안 됨! 다음은 틀린 변환:

if (a > 0)	t = b + c	← 여기서 b+c 계산 (이전 b)
b = 1	if (a > 0)	
d = b + c	b = 1	← b가 바뀔!

$$e = b + c$$

$$\begin{aligned} d &= t \\ e &= t \end{aligned}$$

← t는 옛날 b+c! 틀림!
← 역시 틀림!

b=1이 b를 바꾸는데, 삽입점에서의 b+c는 바뀌기 전 값 → 프로그램 결과 변경
→ unsafe! 이것을 방지하려면 “어디에 삽입해도 안전한가?”를 판별하는 별도의
데이터 흐름 분석이 필요하다 → **Anticipation Analysis**.

8. Anticipation Analysis — “이 식이 앞으로 반드시 계산될 것인가?”

표현식 E가 지점 P에서 **anticipated**하다 = P에서 시작하는 모든 실행 경로에서 E가 계산되며, 그 계산까지 E의 피연산자가 재정의되지 않는다.

Available Expression과 정반대: - Available: “과거에서 P까지 모든 경로에서 E가 계산됨” → **Forward** - Anticipated: “P에서 미래의 모든 경로에서 E가 계산될 것” → **Backward**!

$\wedge = \cap$ (모든 경로에서 계산되어야 함). Backward 방향. 전달함수: $IN[b] = GEN[b] \cup (OUT[b] - KILL[b])$.

PRE에서 삽입이 안전한 조건: 삽입 지점에서 해당 표현식이 **anticipated**여야 한다. 즉, “여기서 미리 계산해도, 어차피 나중에 반드시 같은 값으로 계산될 것이므로 안전”이 보장되어야 한다.

9. AE 프레임워크 성질

분배적(distributive) → MFP = MOP. 반복 알고리즘이 최선의 해를 준다.

Ch6. Control Flow Analysis and Loops — 루프를 찾아라

1. 왜 루프를 찾아야 하는가

루프 최적화(LICM, Strength Reduction, IV Elimination)가 가장 큰 성능 향상을 준다. 프로그램 실행 시간의 대부분이 루프 안에서 소비되기 때문이다. 그런데 컴파일러가 보는 것은 소스 코드가 아니라 **CFG**이다. CFG에서는 for, while, do-while, goto로 만든 루프가 모두 같은 형태의 간선으로 보인다. 그래서 그래프 이론적으로 루프를 정의하고 찾는 방법이 필요하다.

또 하나 중요한 점: 모든 사이클(**cycle**)이 루프는 아니다. 최적화 관점에서 “루프”라고 부를 수 있으려면 조건이 있다.

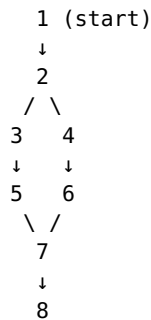
2. 루프의 정의 — 두 가지 조건

루프라고 불리려면: ① 단일 진입점(single entry point) — 외부에서 루프에 들어오는 입구가 딱 하나. 이것을 “헤더(header)”라고 부른다. ② 사이클(cycle) — 간선들이 최소 하나의 사이클을 형성해야 한다. 진입점이 2개 이상이면 최적화용 루프로 보지 않는다.

3. Dominator (지배자)

노드 d가 노드 n을 지배(d dom n)한다 = start 노드에서 n으로 가는 모든 경로가 d를 통과한다. “n에 도달하려면 d를 반드시 거쳐야 한다.” d가 n의 관문(gate)인 셈이다.

슬라이드 예제 CFG:



(+ 역방향 간선: 8→7, 7→2, 5→2 등)

각 노드의 dominator:

노드 1: {1} ← start 자체
노드 2: {1, 2} ← 2에 가려면 반드시 1을 거침
노드 3: {1, 2, 3}
노드 4: {1, 2, 4}
노드 5: {1, 2, 3, 5}
노드 6: {1, 2, 4, 6}
노드 7: {1, 2, 7} ← 3과 4는 dominator가 아님!
1→2→3→5→7 경로와 1→2→4→6→7 경로에서
3을 안 거치는 경로가 존재하므로
노드 8: {1, 2, 7, 8}

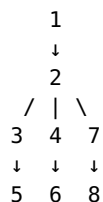
Dominator를 데이터 흐름 분석으로 구하기

도메인: CFG 노드 집합의 부분집합. 방향: Forward. $\wedge = \cap$ (“모든 경로”에서 거치는 노드이므로 intersection). 전달 함수: $OUT[b] = IN[b] \cup \{b\}$ (b를 통과하면 b 자신이 dominator에 추가). 경계: $IN[entry] = \{\}$. 초기화: $OUT[b] = U$ (전체 노드 집합). 분배적 → MFP = MOP.

구체 예: 노드 7의 경우 — $pred = \{5, 6\}$. $OUT[5] = \{1, 2, 3, 5\}$, $OUT[6] = \{1, 2, 4, 6\}$. $IN[7] = \{1, 2, 3, 5\} \cap \{1, 2, 4, 6\} = \{1, 2\}$. $OUT[7] = \{1, 2\} \cup \{7\} = \{1, 2, 7\}$.

Dominator Tree

dominator 관계를 트리로 정리: $a \rightarrow b$ 는 “a가 b를 직접(immediately) 지배”한다는 뜻.



4. Back Edge 찾기

Back edge: 간선 $t \rightarrow h$ 에서 **h가 t를 dominate** ($h \text{ dom } t$). “루프 끝에서 루프 시작(헤더)으로 되돌아가는 간선.”

DFS(깊이 우선 탐색)를 하면 간선을 3가지로 분류할 수 있다:

- Advancing edge: 조상 → 자손 (정방향, 트리 간선)

- Retreating edge: 자손 → 조상 (역방향, 위로 올라감)
- Cross edge: 오른쪽 → 왼쪽 (트리의 다른 가지)

관계: **back edge $\subseteq$ retreating edge** (모든 back edge는 retreating이지만, 역은 항상 성립하지 않음)

대부분의 실제 프로그램(모든 구조적 코드 + 대부분의 goto 프로그램)에서는 retreating edge = back edge. 이런 그래프를 **Reducible Flow Graph**라고 부른다. Reducible graph는 안쪽 루프부터 하나의 노드로 축소할 수 있다.

예: 간선 $8 \rightarrow 7$ 이 back edge인지? 8의 dominator = $\{1, 2, 7, 8\}$. $7 \in \text{dom}(8)$ → YES! 7이 8을 dominate → back edge.

5. Natural Loop 구성

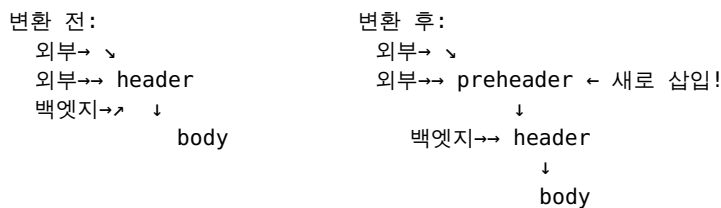
Back edge $t \rightarrow h$ 의 natural loop:

1. h(헤더)를 그래프에서 제거
2. t에서 역방향으로 도달 가능한 모든 노드 찾기
3. 그 노드들 + h = natural loop

두 루프가 같은 헤더를 공유하면 하나로 합침. Inner loop = 다른 루프를 포함하지 않는 가장 안쪽 루프 (최적화의 주요 대상).

6. Preheader — 루프 최적화를 위한 준비

LICM 같은 루프 최적화는 “루프 직전에 코드를 삽입”해야 한다. 루프 헤더에는 외부 간선과 백엠티지가 모두 들어오므로, **preheader BB**를 새로 만든다:



preheader에는 외부 간선만 들어오고, 백엠티지는 header로 유지. LICM으로 이동할 코드를 preheader에 넣으면 루프 진입 전 딱 한 번만 실행된다.

Ch7. Loop Optimizations — 루프 안의 코드를 극한까지 줄인다

이 장은 두 가지 핵심 루프 최적화를 다룬다: LICM(Loop Invariant Code Motion)과 Strength Reduction/IV Elimination.

Part 1: LICM (Loop Invariant Code Motion)

1. 루프 불변 계산이란?

루프 불변 계산(loop-invariant computation) = 루프 안에서 제어가 머무는 동안 값이 변하지 않는 계산. “이 계산은 루프를 100번 돌아도 매번 같은 결과가 나오니까, 루프 밖에서 한 번만 계산하면 된다.”

예:

```

for (i = 0; i < 100; i++) {
    x = a + b;           ← a, b가 루프에서 안 바뀌면 → loop
}
  
```

```
invariant!
    A[i] = x * i;
}
```

변환 후:

```
x = a + b;                ← preheader로 이동! (1번만 계산)
for (i = 0; i < 100; i++) {
    A[i] = x * i;
}
```

2. Loop Invariant 탐지 알고리즘

Reaching Definitions(Ch3)를 사용해서 탐지한다.

반복:

문장 $A = B + C$ 가 loop invariant인 조건:

B에 대해: B의 모든 reaching definitions가 루프 밖에 있거나,
루프 안의 reaching definition이 딱 하나이고 그것이 이미

invariant

C에 대해: 같은 조건

변화가 없을 때까지 반복!

왜 반복하는가? 단계적으로 invariant이 드러나기 때문:

```
preheader
  ↓
header
  /   \
A = B+C (I)    E = 3    (I)    ← 1차: B,C 모두 루프 밖 →
invariant!
F = A+2 (I)                ← 2차: A가 invariant이므로 A+2도!
  \   /
D = A+1
  ↑ (back edge)
```

1차: $A=B+C$, $E=3$ 이 invariant

2차: $F=A+2$ 가 invariant (A 가 1차에서 invariant)

3차: 변화 없음 → 종료

3. Code Motion 조건 — invariant이라고 다 옮길 수 있는 게 아니다!

3가지 조건을 모두 만족해야 preheader로 이동 가능:

조건 1: 해당 문장의 BB가 루프의 모든 exit을 dominate

왜? 루프 본체가 실행 안 될 수도 있기 때문!

```
for (i = 0; i < 0; i++) {    ← 조건 거짓 → 본체 실행 안 됨
    x = a + b;              ← 원래 실행 안 되는 코드
}
```

print(x);

preheader로 옮기면 → 원래 실행 안 됐어야 할 계산이 무조건 실행 → unsafe!

조건 2: 같은 변수를 정의하는 다른 문장이 루프 안에 없어야

Loop: def A; use A; def A; use A; Goto Loop

A의 첫 번째 정의만 invariant이라고 옮기면

→ 루프 안 두 번째 def A가 의미 변경 → unsafe!

조건 3: A의 모든 사용이 해당 정의에 의해 dominate

def A; ... Loop: use A; ... def A(invariant); Goto Loop

옮기면 → 첫 반복의 use A가 루프 밖 def A 대신 preheader의 def A를 사용 → 값 변경!

4. Landing Pad

while 루프를 test-repeat-until 형태로 생성하면 dominance 조건 충족이 쉬워진다:

while (p) DO s → if (p) { preheader; repeat s until not p }

Part 2: Induction Variable Elimination (Strength Reduction)

5. 유도 변수(Induction Variable) 정의

Basic Induction Variable (BIV): 루프 내에서 유일한 정의가 $X = X + c$ 또는 $X = X - c$ 형태인 변수. c 는 상수 또는 loop-invariant.

Induction Variable: BIV 자체이거나, 루프 내에서 딱 한 번 정의되고 그 값이 어떤 BIV B의 선형 함수 $A = c1 \times B + c2$.

예:

```
i = 0;
L2: if (i >= 100) goto L1
    t1 = 4 * i          ← t1은 IV! (t1 = 4×i + 0)
    t2 = &A + t1        ← t2는 IV! (t2 = 4×i + &A)
    *t2 = 0
    i = i + 1           ← i는 BIV
    goto L2
```

6. Strength Reduction 알고리즘 — 곱셈을 덧셈으로

IV A가 $A = c1 \times B + c2$ 형태일 때:

1. 새 변수 A' 생성
2. Preheader에서 초기화: $A' = c1 \times B + c2$ (초기값)
3. B가 변경될 때 ($B = B + x$)마다: $A' = A' + x \times c1$ (곱셈→덧셈!)
4. 원래 A의 정의를: $A = A'$ 로 교체

구체 예 단계별:

원본:

```
i = 0;
L2: if (i >= 100) goto L1
    t1 = 4 * i          ← IV: t1 = 4×i + 0
    t2 = &A + t1        ← IV: t2 = 4×i + &A
    *t2 = 0
    i = i + 1           ← BIV
    goto L2
```

Step 1: 새 변수 + preheader 초기화

```
t1' = 4 * 0 = 0
t2' = 4 * 0 + &A = &A
t3 = 4 * 100 + &A    ← 루프 상한 (LFTR용)
```

Step 2: BIV 변경($i=i+1$) 뒤에 업데이트 추가

루프 내: $t1 = t1'$; $t2 = t2'$; $*t2 = 0$; $i = i+1$; $t1' += 4$; $t2' += 4$

Step 3: 후속 최적화

Copy propagation: $t1=t1' \rightarrow t1$ 사용처를 $t1'$ 으로 $\rightarrow t1$ dead $\rightarrow$ 삭제
 $t2=t2' \rightarrow *t2'=0$

Step 4: IV Elimination (LFTR)

루프 조건 $i \geq 100 \rightarrow t2' \geq t3$ 으로 교체
 $i=i+1$ 은 dead $\rightarrow$ 삭제, $t1'+=4$ 도 dead $\rightarrow$ 삭제

최종:

```
t2' = &A
t3 = &A + 400
L2: if (t2' >= t3) goto L1
    *t2' = 0
    t2' = t2' + 4
    goto L2
```

$\rightarrow$ 곱셈 완전 제거, BIV i 도 제거. 루프 본체 3개 명령어!

7. Non-trivial BIV — 정의가 여러 개인 경우

BIV의 정의가 여러 개(조건부 포함)여도 Strength Reduction이 가능하다. 핵심: 각 BIV 정의 바로 뒤에 해당 IV의 업데이트를 추가.

```
k = 0; m' = 2*k = 0;      ← preheader
for (...) {
    k = k + 3; m' = m' + 6;  ← k+=3이니까 m'+=2*3=6
    if (x<y) { k = k+4; m' = m'+8; }
    if (a<b) m = m';        ← m=2*k를 m=m'으로 교체
    k = k-2; m' = m'-4;     ← k-=2이니까 m'-=2*2=4
}
```

Ch8. Register Allocation — 변수를 레지스터에 배정하기

1. 문제의 중요성

프로그램에서 사용하는 변수(의사 레지스터)는 수백~수천 개인데, CPU의 물리 레지스터는 보통 16~32개뿐이다. 어떤 변수를 어떤 레지스터에 배정할 것인가? 레지스터에 배정 못 받으면 메모리(스택)로 가야 하는데, 이것은 100배 이상 느리다. 따라서 레지스터 할당은 성능에 가장 직접적인 영향을 주는 최적화이다.

기본 용어: Allocation(레지스터에 유지 결정), Assignment(구체적으로 어떤 레지스터에), Spilling(레지스터 배정 못 받아 메모리로).

2. 간섭(Interference) — 언제 같은 레지스터를 쓸 수 없는가?

두 의사 레지스터가 프로그램의 어떤 지점에서 **동시에 live**하면 간섭한다. 두 변수가 동시에 살아있으면 서로 다른 레지스터를 써야 한다.

a = ...	a live	■■■■■■■■■■
b = ...	b live	■■■■■■■■■■
= a + b	← 여기서 a, b 동시에 live $\rightarrow$ 간섭!	
c = ...	c live	■■■■■■■■■■
= b + c	← b, c 동시에 live $\rightarrow$ 간섭!	
	← a, c는 동시에 live인 구간 없음 $\rightarrow$ 간섭 안 함 $\rightarrow$ 같은	

레지스터 가능!

3. 간섭 그래프(Interference Graph)

노드 = 의사 레지스터(또는 live range), 간선 = 동시에 live인 쌍. 레지스터 할당 = **그래프 n-컬러링** (n = 물리 레지스터 수). 간선으로 연결된 두 노드는 다른 색이어야 함. n-컬러링은 **NP-complete** → 최적 해를 보장하는 다항 시간 알고리즘 없음 → 휴리스틱 사용.

4. Live Range (Web) — 간섭 그래프의 노드를 정밀하게

같은 변수라도 서로 다른 시점에 정의된 값이 서로 다른 레지스터에 들어갈 수 있다. Live range = 정의 d에서 시작하여, d가 도달(RD)하면서 변수가 live(LV)인 모든 지점. 같은 변수의 겹치는 live range는 merge. 분리하면 간섭 그래프가 더 성가게(sparse) → 컬러링이 쉬워진다.

간섭 구축(definition-oriented): 각 명령어 I에서, I가 정의하는 live range x와, I의 끝에서 live인 모든 live range y 사이에 간섭 삽입.

5. Chaitin's Coloring Algorithm — degree < n 규칙

핵심 관찰: **degree < n**인 노드는 **항상 색칠 가능!** 이웃이 n-1개 이하이므로, n개 색 중 남는 색이 최소 1개.

[Simplify]

반복: degree < n인 노드 v를 스택에 push, 그래프에서 제거
(제거하면 이웃 degree 감소 → 새로 degree < n 발생 가능!)
모든 노드 제거되면 → 컬러링 가능!

[Assign] (역순)

스택에서 pop, 그래프에 다시 삽입, 이웃과 다른 색 배정 (반드시 가능!)

6. Spilling — 레지스터 부족 시

모든 노드의 degree $\geq n$ 이면 stuck. 어떤 노드를 메모리로 spill해야 한다.

Spill 비용: 사용/정의 횟수 $\times$ 루프 중첩 가중치 ($c \times 10^d$). Spill 이득: degree (이웃이 많을수록 효과 큼). cost/degree가 낮은 노드를 우선 spill.

Spill 후: 해당 live range에 def 직후 store, use 직전 load 삽입. 간섭 그래프 재구축 → Simplify 반복.

7. Optimistic Coloring (Briggs) — 더 낙관적으로

Chaitin은 degree $\geq n$ 이면 바로 spill 결정(비관적). 하지만 degree $\geq n$ 이라도 이웃들이 모두 다른 색을 쓰지는 않을 수 있어서 실제로는 컬러링 가능할 수 있다.

Briggs: degree $\geq n$ 이어도 일단 스택에 넣고 나중에 색칠 시도. Assign에서 실제로 색이 없으면 그때 spill. Chaitin이 컬러링하는 모든 그래프를 컬러링 가능 + 일부 추가 그래프도 가능. spill이 필요하면 Chaitin보다 같거나 적은 수만 spill.

슬라이드 예($n=3$): 모든 노드 degree ≥ 3 → Chaitin은 stuck → spill. Briggs는 하나를 일단 push → degree 감소 → 연쇄 simplify → Assign에서 3색 성공!

8. Live Range Splitting

spill 대신 live range를 쪼개서 간섭 줄이기.

Case 1: 거의 죽은 구간(nearly dead zone) 분리 — 두 변수가 전체적으로는
동시 live이지만 각 구간에서는 한쪽만 사용. 경계에서 store/load 삽입하여 같
은 레지스터를 번갈아 사용.

Case 2: copy 삽입으로 다른 레지스터 배정 — live range 중간에 copy를 넣어
다른 레지스터로 옮기면 동시 live 구간을 피할 수 있음. copy 1개 비용으로
spill 회피.

9. Copy Coalescing

COPY x,y에서 x와 y가 간섭하지 않으면 합쳐서 같은 레지스터 배정 → COPY
삭제. 단, 합치면 degree 증가 → 컬러링 어려워질 수 있음.

전략 비교: Aggressive(무조건 합침, copy 최대 제거, degree 폭증 위험) /
Conservative(degree 안 늘면만 합침, 안전) / Iterated(simplify↔coalesce
반복, 더 많은 기회) / Optimistic(aggressive+spill시 분리)

10. Caller-Save vs Callee-Save 레지스터

함수 호출 시 레지스터 값 보존 문제. Caller-save: 호출자가 호출 전 store, 후
load. 호출 경계를 넘어 live한 변수에 배정하면 루프 안 호출시 비용 큼. Callee-
save: 피호출자가 entry에서 store, exit에서 load. 함수 전체에서 1번만.

최적 전략: 호출 경계 안 넘는 변수 → caller-save. 넘는 변수 → callee-save.
Leaf 함수(호출 안 하는 함수)는 caller-save만으로 충분 → 저장/복원 불필요.

11. Pre-colored 레지스터

SP, FP, GP, 함수 인자, 리턴값 등 미리 물리 레지스터가 고정된 변수. 간섭 그래
프에 pre-colored 노드로 넣고, copy coalescing으로 자연스럽게 처리.

Ch11. Static Single Assignment (SSA) Form — 변수에 번호를 매겨서 분석을 쉽게

1. SSA란?

SSA = 각 변수가 프로그램 텍스트에서 **딱 한 번만** 정의되는 형태. 같은 변수가
여러 번 대입되면 각각 다른 이름을 부여한다($a_1, a_2, a_3, \dots$).

일반 형태:	SSA 형태:
$a = x + y$	$a_1 = x + y$
$b = a - 1$	$b_1 = a_1 - 1$
$a = y + b$ ← 재정의	$a_2 = y + b_1$ ← 다른 이름!
$b = x * 4$ ← 재정의	$b_2 = x * 4$
$a = a + b$	$a_3 = a_2 + b_2$

“Static”의 의미: 프로그램 텍스트에서 한 번. 루프 안이면 런타임에 여러 번 실행
될 수 있지만 소스 코드상으로는 한 줄.

2. 왜 SSA가 필요한가?

일반 형태에서 “x”라는 이름만으로는 어느 정의에서 온 값인지 즉시 알 수 없다.
Reaching Definitions 같은 분석이 필요하다.

일반 형태:	SSA 형태:
$x = 1$	$x_0 = 1$

$= x + 10 \leftarrow \text{어느 } x?$	$= x_0 + 10 \leftarrow x_0! \text{ 확실!}$
$x = 5$	$x_1 = 5$
$= x + 10 \leftarrow \text{어느 } x?$	$= x_1 + 10 \leftarrow x_1! \text{ 다른 것!}$

SSA의 3가지 장점: - 모든 use가 이름으로 def에 직접 연결 $\rightarrow$ **RD 분석 불필요!**
 - Available Expression에서 **KILL**이 비어짐 (변수 재정의 없으므로) - Def-Use 체인: $O(N \times M) \rightarrow O(N+M)$

3. 제어 흐름이 없는 경우 — 쉬움

분기 없으면 각 정의에 새 이름 붙이고 이후 사용에 전파하면 끝.

4. 제어 흐름이 있는 경우 — ϕ -function

분기 후 합류점에서 “어느 경로로 왔느냐에 따라 다른 변수를 선택”해야 한다:

일반:	SSA:
$b = M[x]$	$b_1 = M[x_0]$
$a = 0$	$a_1 = 0$
if $b < 4$	if $b_1 < 4$
$a = b$ $\rightarrow$	$a_2 = b_1$
$c = a + b$	$a_3 = \phi(a_2, a_1) \leftarrow \text{경로에 따라 선택!}$
	$c_1 = a_3 + b_1$

ϕ -function의 핵심 성질: - 표기법(notational fiction): 실제로 실행되는 함수가 아님! 분석/최적화용 표기. - 경로에 따라 값 선택: $\phi(a_2, a_1)$ 에서 첫 인자는 첫 predecessor 경로, 두 번째는 두 번째. - 합류점의 시작에 배치.

5. Dominance Frontier — ϕ 를 어디에 삽입할 것인가?

모든 합류점에 ϕ 를 넣으면 너무 많고 비효율적. 필요한 곳에만 넣어야 한다.

Dominance Frontier of x , $DF(x) = \{w \mid x \text{가 } w \text{의 어떤 predecessor를 dominate하면서, } x \text{가 } w \text{를 strictly-dominate하지 않는 노드}\}$

직관: x 가 지배하는 영역의 경계. “ x 의 영향력이 끝나고, 다른 경로와 만나는 지점.”

ϕ 삽입 규칙: 변수 a 를 정의하는 BB x 에 대해, $DF(x)$ 의 각 노드에 a 의 ϕ 를 삽입.

이름 변경(Renaming): Dominator tree를 BFS로 방문, 각 use를 dominator tree에서 가장 가까운 def의 이름으로 교체.

6. SSA 기반 최적화 — 상수 전파 예시

일반 형태:	SSA 형태:
$x = 1$	$x_0 = 1$
$y = 2$	$y_0 = 2$
$z = x + y \leftarrow \text{어떤 } x?$	$z_0 = x_0 + y_0 \leftarrow x_0=1 \text{ 확실!} \rightarrow z_0=3$
if (...)	if (...)
$h = x * 2 \leftarrow \text{어떤 } x?$	$h_0 = x_0 * 2 \leftarrow x_0=1 \text{ 확실!} \rightarrow h_0=2$
$x = f(\dots)$	$x_1 = f(\dots)$
$k = x - y \leftarrow \text{어떤 } x?$	$x_2 = \phi(x_0, x_1)$
	$k_0 = x_2 - y_0 \leftarrow x_2 \text{는 불확실}$

7. SSA $\rightarrow$ 일반 형태 (Back Transformation)

최적화 후 기계어 생성을 위해 되돌린다. $\phi(x_1, x_2)$ 를 각 predecessor 끝에 **copy 명령어**로 변환.

Lost Copy Problem: Copy propagation 후 back transform하면 순서 문제 발생 가능.

원본: $x=1; y=x; x=2; \text{return } y$ ($y=1$ 반환)
SSA: $x_0=1; x_1=\phi(x_0, x_2); y_1=x_1; x_2=2; \text{return } y_1$
Copy prop: $x_1=\phi(x_0, x_2); x_2=2; \text{return } x_1$
Back transform: $x_1=x_0; x_2=2; x_1=x_2; \text{return } x_1$ ← 항상 2 반환!
INCORRECT!

해결: **Edge-Split SSA Form** — 간선 $a \rightarrow b$ 에서 a 의 successor가 2개+ AND b 의 predecessor가 2개+이면, 빈 노드를 사이에 삽입. ϕ 의 copy를 전용 BB에 배치 → 충돌 방지.

8. SSA는 업계 표준

LLVM 내부 IR이 SSA. DCE, CSE, LICM, 상수 전파, value numbering 등 모두 SSA 기반. Android Dalvik VM, Java HotSpot VM, V8 JavaScript Engine도 SSA 사용.

Ch9. Instruction Scheduling — 명령어를 재배치하여 CPU를 쉬지 않게

1. 왜 명령어 스케줄링이 필요한가

현대 CPU는 파이프라인으로 동작한다. 명령어 순서가 안 좋으면 stall 발생. 스케줄링 = 독립적 명령어 재배치로 stall 제거 + CPU 자원 최대 활용.

2. Parallel Group

superscalar/VLIW CPU는 한 사이클에 여러 명령어 동시 실행. 매 사이클 독립적 명령어를 묶어 동시 실행. 자원이 차거나 의존성으로 못 넣으면 다음 사이클.

3. 두 가지 제약

1. 자원 제약: 실행 유닛 수 제한 (예: ALU 1+MEM 1).
2. 의존성 제약: True dependence(RAW, write→read, 진짜, delay>0), Anti dependence(WAR, read→write, 이름 충돌, delay=0), Output dependence(WAW, write→write, 이름 충돌). Anti/Output는 renaming 해소.

4. 의존성 극복

Speculative code motion(제어 의존성): 분기 전에 미리 실행. 3조건 — 정확성(새 변수), 예외무발생, 영구변경안함(store 불가).

On-demand renaming(Anti/Output): 새 레지스터 이름 도입. Forward substitution(copy의 true dep): copy 제거 후 직접 사용.

Memory disambiguation: store/load 주소 비교. No→이동. Yes→store-copy. Maybe→unsafe_load+recovery.

5. List Scheduling (BB 스케줄링)

DAG 구성: 노드=명령어, 간선=의존성(delay). READY=선행자 0인 노드. 반복: 우선순위 최고 선택 → 가장 빠른 슬롯 배정. 우선순위: critical path > delay > source order. NP-complete → 휴리스틱.

예제: i1:LD r2=9(r1), i2:ST r4,0(r3), i3:ADD r4=r4,r3, i4:ADD r1=r5,r4, i5:ADD r6=r2,r4 (ALU1+MEM1, LD=2cyc). DAG: i1→(2)i5, i2→(0)i3, i3→(1){i4,i5}. 스케줄: Cyc1:[i1,i3], Cyc2:[i2,i4], Cyc3:[i5].

6. Top-down vs Bottom-up

Top-down(AEAP): 빠른 시작, 레지스터 수명 길어짐. Bottom-up(ALAP): 수명 짧음, expression tree 유리.

7. RA와 스케줄링 상호작용

pre-RA(자유도 높) → RA → post-RA(spill 반영).
Integrated(Goodman&Hsu): 레지스터 여유면 병렬성 우선, 부족하면 수명 축소.

Ch10. Software Pipelining — 루프의 여러 반복을 겹쳐서 동시에 실행

1. 왜 소프트웨어 파이프라이닝이 필요한가

Ch9의 BB 스케줄링은 한 반복 안에서만 재배치. 하지만 루프에서는 서로 다른 반복의 명령어끼리 독립적인 경우가 많음. 겹쳐 실행하면 성능이 훨씬 좋아짐.

2. 동기 예제

머신: 매 사이클 MEM 1개 + ALU 1개 동시 시작 가능. MEM latency=1cyc. ALU latency=2cyc이지만 파이프라이닝(매 사이클 새 연산 시작 가능, 결과는 2cyc 후). 서로 다른 연산의 ALU가 겹칠 수 있음(throughput=1op/cyc).
 $A[i] = A[i] * b + c$ 루프.

한 반복: Read(1)→Mul(2)→Add(2)→Write(1) = 6cyc. 매 사이클 절반이 노는 중. 자원 기준 최적: MEM 2개+ALU 2개를 2사이클에 처리 → 2cyc/iter.

3. Loop Unrolling의 한계

루프 본체를 여러 번 복사하여 다른 반복 명령어를 섞음. 효율 90% → 18번 unroll 필요 → 코드 크기 폭발.

4. Software Pipelining — 최적 해

여러 반복을 겹쳐서 정상 상태(Kernel)를 만듦. 예제의 Kernel:
 $[Add(i) + Read(i+2)] \mid [Write(i) + Mul(i+1)] = II = 2cyc/iter = \text{최적! 코드 크기는 원본의 2~3배만 (unrolling 18배 대비 훨씬 적음).}$

5. 파이프라인 루프 구조

Prolog(파이프라인 채움) → Kernel(정상 상태, 매 II사이클 1반복) → Epilog(파이프라인 비움). CPU 파이프라인 (fetch/decode/execute/writeback)과 같은 개념을 소프트웨어로.

II(Initiation Interval) = Kernel의 사이클 길이 = 다음 반복 시작까지 간격. II가 작을수록 좋음.

6. Do-Across 루프

Do-All 루프: 반복 간 독립. Do-Across 루프: 반복 간 의존성(예: $Sum = Sum + A[i]$). Do-Across도 파이프라이닝 가능하지만 II가 더 길어질 수 있음.

7. Modulo Scheduling — 가장 널리 쓰이는 기법

$MII(\text{Minimum Initiation Interval}) = \max(RMII, PMII)$.

$RMII(\text{Resource MII}) = \max(r_i/R_i)$. r_i =한 반복의 자원 i 사용량, R_i =CPU의 자원 i 수. 예: 2-ALU+1-MEM, ALU 4개+MEM 3개 → $RMII = \max(4/2, 3/1) = 3$.

$PMII(\text{Precedence MII})$: 의존성 그래프에서 사이클이 있으면, 각 사이클의 $\text{sum}(\text{delay})/\text{sum}(\text{iteration_diff})$ 의 최대값. 간선 $\langle p, d \rangle$: p =반복 간격, d =지연 사이클.

알고리즘: $II = MII$ 에서 시작. Modulo Reservation Table(II행×자원열)에 배정 시도. 명령어를 사이클 x 에 배정하면 $x, x+II, x+2*II, \dots$ 에서 실행 → $x \bmod II$ 행의 자원 사용. 성공→확정, 실패→ $II+1$ 로 재시도. NP-complete → 시행착오.

8. Modulo Variable Expansion

어떤 변수의 live range가 II보다 길면 다음 반복에서 같은 레지스터를 덮어씀. 해결: 수명>II인 변수에 여러 레지스터 할당 + 커널 unroll. 예: $L(v) = 3 \times II$ → 레지스터 3개. 또는 Rotating Register(Intel Itanium): HW가 brtop 명령어로 레지스터 번호 자동 시프트 → unroll 없이 해결.

9. EPS (Enhanced Pipeline Scheduling)

MS와 완전히 다른 접근. 원래 루프를 유지하면서 DAG 스케줄링을 반복 적용. 명령어가 루프 백엠티를 넘어 이동하면 파이프라이닝 효과 발생. 장점: 임의 제어 흐름 루프, 비상수 반복수, outer loop, 비구조적 루프, variable II(경로에 따라 다른 II) 모두 처리 가능.

Ch12. VM, Interpreter, and JIT Compiler

— 프로그램을 실행하는 다른 방법들

1. 프로그램 실행 방법 3가지 (Ch1 확장)

방법 1: 소스→기계어→HW 실행 (C/C++). 가장 빠르지만 이식성 낮음. 방법 2: 소스→바이트코드(IR)→VM 실행 (Java, Ethereum). 이식성 좋음. 방법 3: 소스→VM이 내부적으로 IR→실행 (JavaScript, Python). 사용자는 소스 직접 실행처럼 보이지만 내부에서 바이트코드 변환.

2. Decode-and-Dispatch 인터프리터

가장 단순. while 루프 + switch/case로 바이트코드를 하나씩 해석. 문제: 매 바이트코드마다 간접 분기 2개(switch→case, return→while). CPU 분기 예측기가 잘 동작 못 함 → 느림.

3. Threaded Interpretation

switch/return 제거. 각 루틴 끝에서 dispatch table 조회 후 다음 루틴으로 직접 goto. 간접 분기 1개/바이트코드로 줄임 → 50%+ 성능 개선.

4. JIT Compilation

바이트코드를 런타임에 기계어로 변환. 핵심: hot 코드만 컴파일 (Hotspot Detection — 함수 호출 횟수 카운트). 컴파일 오버헤드 vs 실행 속도 이득의 트레이드오프. RA: Graph Coloring 대신 Linear Scan (빠르지만 덜 최적). 코드 캐시에 저장 재사용. 서버측 프로그램(수개월~수년 실행)은 컴파일 오버헤드 무시 가능.

5. Multi-tier Architecture (JavaScript 엔진)

1st Tier(cold): Parse → 바이트코드 → 인터프리터 (빠른 시작, 최적화 없음). 2nd Tier(warm): Baseline JIT (1:1 변환, 프로파일링 수집). 3rd Tier(hot): Optimizing JIT (프로파일 기반 CSE/DCE/constant folding/RA, 코드 크기 0.32배). Deoptimization: 최적화 가정이 깨지면 baseline으로 되돌아감.

6. Profile-based Optimization

JavaScript는 동적 타입 → 컴파일 시점에 타입 모름. 프로파일링으로 “이 변수는 항상 integer” 정보 획득 → 타입 특화 코드 생성(int add만 사용). hidden class 캐싱으로 객체 속성 직접 접근. 가정 깨지면 deoptimization handler로 점프.

Ch13. Compiler-Friendly Programming — 프로그래머가 컴파일러를 도울 수 있다

1. 핵심 메시지

같은 빅-O 알고리즘이라도 코드 작성 방식에 따라 10배 성능 차이. 프로그래머가 컴파일러 최적화를 이해하고, 최적화를 방해하는 패턴을 피하면 성능 대폭 향상.

2. 프로그래머 LICM — strlen() 문제

느린: for(i=0; i<strlen(s); i++) → strlen() 매 반복 O(N) → 전체 O(N²). 빠른: int len=strlen(s); for(i=0; i<len; i++) → O(N). 왜 컴파일러가 자동으로 못 옮기나? strlen()이 부작용 가능(전역변수 변경 등) → 컴파일러는 함수를 블랙박스 취급.

3. 프로그래머 Strength Reduction

매 반복 n*i 곱셈 → ni=0; ni+=n 덧셈으로 교체. 컴파일러가 자동으로 하기도 하지만(-O1), 복잡한 경우 프로그래머가 직접.

4. 프로그래머 CSE

$(i-1)n, (i+1)n, in$ 곱셈 3번 $\rightarrow inj=in+j$ 계산 후 $inj-n, inj+n, inj-1, inj+1$ 로 교체. 곱셈 1번으로 줄임.

5. Optimization Blocker #1: 함수 호출

컴파일러는 함수=블랙박스. 부작용 가정 $\rightarrow$ LICM/CSE 등 적용 제한. 해결: inline 함수, 프로그래머 직접 코드 이동.

6. Optimization Blocker #2: Memory Aliasing

a,b가 같은 메모리 가리킬 수 있으면(aliasing) 컴파일러가 $b[i]$ 를 지역변수로 교체 못 함. 예: `double B[3]=A+3`이면 $b[i]$ 쓰기가 $a[]$ 값을 바꿈! 해결: 프로그래머가 지역변수에 누적 후 최종 저장. “컴파일러에게 aliasing 안 한다고 알려주는 것”과 같은 효과.

7. 성능 비교

combine1(무최적화): 22.68 $\rightarrow$ combine1(-O1): 10.12 $\rightarrow$ combine4(프로그래머 최적화): 1.27. 약 18배 개선. 핵심: `vec_length` 루프 밖, `bounds check` 제거, 지역변수 누적, 포인터 직접 접근.